

Verifying a Virtual Machine Monitor

PAOLO G. GIARRUSSO, BlueRock Security, Inc, Germany

HOANG-HAI DANG, BlueRock Security, Inc, Germany

FRANTIŠEK FARKA, BlueRock Security, Inc, Germany

GREGORY MALECHA, BlueRock Security, Inc, USA

Virtualization provides a core technology that underlies much of the modern computing infrastructure. Virtual Machine Monitors (VMMs) multiplex physical hardware enabling it to run multiple systems. However, virtualizing a full computer system is complex and the virtualization platform is necessarily highly privileged and therefore a prime target for attack. We present our approach to specifying and verifying the correctness of the BlueRock VMM. This VMM is written in C++ and runs on top of the NOVA microhypervisor. We show how the VMM architecture naturally aligns with the specification of a machine as distributed systems, and how we can capture this alignment naturally within separation logic. Within this architecture, we focus on the modular verification of CPU cores which are implemented using a combination of hardware virtualization and software emulation.

Additional Key Words and Phrases: Virtualization, verification, Separation logic, Iris, NOVA, virtual machine monitor

1 INTRODUCTION

Virtual machines are a ubiquitous abstraction in modern software systems. They enable running multiple operating systems in much the same way that operating systems enable running multiple applications. The highly-privileged nature of virtual machine monitors (VMMs)—software that runs virtual machines—makes them high-value targets for attack. At the same time, the low-level and highly-concurrent nature of the code makes them difficult to get right. We aim to provide a trustworthy foundation for virtualization by proving the correctness of the BlueRock VMM running within the BlueRock ULTRA platform. In addition to ensuring the integrity of the ULTRA platform, the proof will also guarantee that the guest runs on architecture-conformant (virtual) hardware.

To achieve these properties, we aim for a refinement specification for the VMM called the bare-metal property[™]. This specification states that *for any* boot image, e.g., a kernel image of the guest OS, the behavior of the VMM running that boot image is a subset of the possible behaviors of an architecture-conforming, bare-metal machine running that same image. Capturing this specification requires a *full computer system* model, not just a CPU ISA description. This model must include memory, interrupt controllers, bridges and buses, as well as hardware devices such as serial devices, network cards, etc. In this work, we focus on the ARMv8 architecture, but the BlueRock VMM is designed to support multiple architectures (using different compilation options).

We present our on-going mechanized correctness proof of the BlueRock VMM, an industrial-strength VMM. Our work to date establishes the verification architecture of the VMM implementation, connecting the program-level abstractions to the abstractions from the specification. Our verification builds on the rich separation logic specification of the NOVA microkernel [15, 6], which the VMM runs on top of. We carry out our verification using the BRiCK program logic for C++ [9], leveraging the BlueRock separation logic automation.

In this work, we focus on the subtle connection between hardware virtualization and software emulation for CPUs. We report several core innovations in our work, centering around the modeling of the hardware and the architecture of the proof. Specifically,

- (1) We present a model of machines as distributed systems, including multiple cores (CPUs), memory, devices, and interrupt controllers. We use this model as the specification of a virtual machine monitor. (§3)

Authors' addresses: Paolo G. Giarrusso, BlueRock Security, Inc, Germany, paolo@bluerock.io; Hoang-Hai Dang, BlueRock Security, Inc, Germany, hai@bluerock.io; František Farka, BlueRock Security, Inc, Germany, frantisek@bluerock.io; Gregory Malecha, BlueRock Security, Inc, USA, gregory@bluerock.io.

- (2) We demonstrate how this modular machine model enables us to similarly decompose the verification of the VMM into independent components that can be verified separately. (§4)
- (3) We demonstrate how the NOVA specification style enables deep reasoning about programs running on NOVA, and thus enables us to reason uniformly about hardware virtualization, software emulation, and the transition between these. (§5)
- (4) We demonstrate how separation logic enables separate verification of the system’s components, including communication and coordination between these components. (§6, §7, §8, §9)

2 BACKGROUND

Before diving into the details of the VMM verification, we set the stage by discussing relevant background. The BlueRock VMM runs within the BlueRock ULTRA platform on top of the NOVA microhypervisor which abstracts some of the very low level hardware details and exposes slightly higher level abstractions. To reason about code running on top of NOVA, we use the NOVA specification [6], which captures the behavior of NOVA in separation logic (Section 2.1), and reflects CPU semantics in a way that simplifies reasoning (Section 2.1.1). On top of NOVA, the VMM itself is implemented in systems-level, concurrent C++ leveraging features introduced up through C++20. We carry out its verification using BlueRock’s BRiCK separation logic-based program logic for C++ [9] (Section 2.2). As the top-level property that we aim to prove is a refinement property, we review the approach to proving refinements in separation logic (Section 2.3).

2.1 The NOVA Microhypervisor

The NOVA microhypervisor [15, 13] is a modern, highly-concurrent, capability-based microkernel, which runs in hypervisor mode to support hardware virtualization. Following the microkernel design philosophy, NOVA provides minimal security-required abstractions and exposes functionality to user-space programs that execute on top of it, either directly or through abstractions. For example, NOVA exposes the following capability-based APIs [14, §4.1] for interacting with the physical system:

- Page tables are exposed through protection domains (PDs). There are separate components for user-space page tables (host), guest page tables (guest), and DMA page tables (device).
- The CPU state is exposed through execution contexts (ECs) which encapsulate CPU register files. ECs are further sub-divided into three types:
 - Global ECs represent user-level threads.
 - vCPU ECs represent guest-level threads, *i.e.*, those that have virtualized “kernel mode” execution.
 - Local ECs are non-active threads that are used to respond to inter-process communication (IPC) calls.
- Synchronous communication between threads is captured by inter-process communication which is exposed through portals (PTs). Portals have local state corresponding to an instruction pointer to jump to, and are attached to a local execution context that captures the execution environment. In this work, we will primarily focus on architectural portals [14, §2.4], which NOVA uses to handle faults, *e.g.*, a page fault or a division by zero, by other ECs.
- Execution time on CPUs is captured by scheduling contexts (SCs).
- Interrupts, and other inter-core communication is exposed via semaphores (SMs).

BlueRock’s formal specification of NOVA [6] captures the state and behaviors of NOVA using the Iris separation logic [8]. It exposes the state of NOVA as separation logic ownership. For example, the value and wait queue of a semaphore is exposed through two predicates $\text{sm.value } \gamma_{sm} \ n$ and $\text{sm.queue } \gamma_{sm} \ \text{ls}$ where γ_{sm} represents the kernel name of the semaphore. The behavior of NOVA is reflected as a weakest (liberal) pre-condition for each global and vCPU EC. We discuss these in more detail in Section 5.1.2 as they play a central role in the VMM proof.

2.1.1 Decomposing the CPU Semantics. A crucial feature of the NOVA specification is the way that it directly enables user-space reasoning. The specification is based on a formal decomposition of the architectural CPU semantics into different privilege levels.

While proving that a vCPU refines the bare-metal CPU, we would like to avoid the case distinction on all instructions, where we would need to show refinement for both virtualized and non-virtualized instructions. For modern architectures with large instruction sets, this case analysis is prohibitively expensive. Furthermore, correctness of hardware virtualization is mainly the responsibility of the ISA architects, not the VMM implementer.

To facilitate reasoning about how virtualization state affects the behavior of the CPU, we want a CPU model that allows us to model virtualization state precisely while abstracting away from the semantics that are not virtualization-relevant, *e.g.*, arithmetic operations. Decomposing the processor state into virtualization-relevant and virtualization-irrelevant state allows us to precisely model the former while abstracting away the latter. In other words, we only care about the difference between the bare-metal CPU model and the CPU model used in the implementation. Virtualization-irrelevant behaviors are (parametrically) the same in both models, and their simulation is trivial.

The semantics of CPUs is further complicated by the fact that software can observe that the CPU does not execute instructions one by one logically atomically. As a first step, we follow the approach by Armstrong et al. [4] to describe the CPU semantics (RTL core) using a combination of two pieces:

- (1) An *imperative semantics* for each instruction in isolation (**ASL/SAIL**) that describes how the state is updated and when communication occurs with the rest of the system, *e.g.*, when accessing memory. We generally abuse terminology and call this the ASL or SAIL models of the CPU when in reality ASL and SAIL are the programming languages used to describe these semantics.
- (2) An *instruction speculator and scheduler* (**RMEM**) that interleaves the execution of multiple instructions and manages state between them. This extra level of scheduling is necessary because the imperative instruction semantics does not account for the advanced degree of concurrency allowed in modern microprocessors, *e.g.*, out-of-order loads and speculations that are allowed by the processor. Consequently, the semantics of CPUs is described as a tree of instructions (ToI) whose branches are executed in parallel.

It is worth noting that there are several different semantic styles for writing the “scheduler” component. The work here is derived most closely from the tree-of-instructions model, but the ideas here might be transportable to the other semantic styles with different trade-offs.

In the second step, we extend the SAIL model with the effects of accessing virtualization-relevant states. In the work of Armstrong et al. [4], only memory operations are exposed as events. We add events for accessing other virtualization-relevant states. These virtualization effects are interpreted by event handlers that are still internal to the processor, as shown in the diagram in Figure 1. Virtualization-relevant events from the RTL core are either handled using the internal state of the handler producing internal steps (τ -steps) for the full processor model, or are translated to external events such as memory accesses using the virtualization state, *e.g.*, 2nd-stage page tables. In case of the bare-metal CPU model, the handler follows the architectural semantics without virtualization extensions. In case of the implementation, the handler follows the virtualization semantics.

Example: Control Registers. To better understand the decomposition, consider the virtualization flag on ARM that enables trapping on changes to the page table base register (TTBR0). To ensure that the virtualization handler gets access to all changes to this register, we remove TTBR0 and the flag from the RTL core state and only provide access to them via get and set events. The bare-metal handler (with no virtualization extensions) simply handles these events by manipulating its own value of TTBR0. The virtualized handler would instead consult the flag and inform the RTL core that it must not complete the write, but rather trap. If the write was executed out of order, the RTL core will defer the trap appropriately to emulate in-order execution.

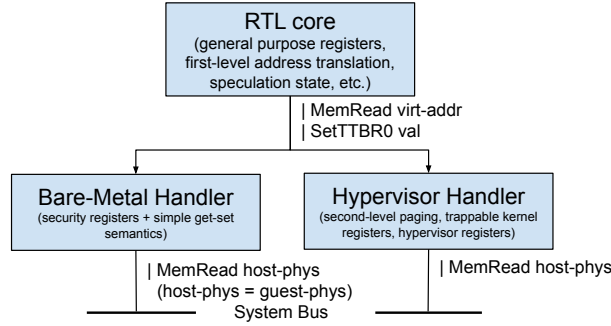


Fig. 1. Decomposition of the processor model to isolate virtualization-relevant state.

This decomposition eases reasoning precisely about the behavior of the flag, while abstracting the details of which instructions actually produce the relevant events. To prove virtualization of TTBR0 correct, we prove that “TTBR0 emulation works”: our VMM’s software handler for trapped accesses to TTBR0 refines untrapped accesses to TTBR0 in the non-virtualized semantics.

Example: Memory. Memory accesses in virtualized CPUs add extra steps compared to unvirtualized CPUs, which we encapsulate in effect handlers. Virtualized CPUs have two stages of address translation. The first stage translates guest virtual addresses (GVAs) to guest physical addresses (GPAs) or invokes the guest page-fault handler. This is guest behavior and is encapsulated entirely in the RTL core. The second stage is exclusive to virtualized CPUs and translates GPAs to host physical addresses (HPAs), or faults to the hypervisor. This translation is implemented by the handler for virtualized systems. For unvirtualized systems, the translation is the identity.

To prove virtualization of memory correct, we prove that “memory emulation works”: accessing virtualized memory refines accessing non-virtualized memory. If the guest is reading GPA $0x4000$, the VMM must ensure the 2nd-stage translation maps that address to a host address with the correct value, or the VMM must trap the access and emulate it, for example by probing a virtual device.

Justifying this split is more subtle: in hardware, page table translations for both stages are carried out by the same memory management units (MMUs) and results are cached by translation lookaside buffers (TLBs). Yet, we abstract from the hardware and split TLBs and MMUs between the RTL core and event handler: the RTL core handles 1st-stage translation and encapsulates associated TLB state for 1st-stage page tables, while the event handler implements 2nd-stage translation and includes associated TLB state for 2nd-stage page tables.

2.1.2 Decomposing the CPU Simulation. We look into more details on the decomposition of the simulation between the bare-metal CPU and the physical CPU specifications (described in the ASL model), shown in Figure 2.

The refinement connects the two red boxes. The bottom-right box is the implementation’s physical CPU, while the top-left is the bare-metal CPU specification, which is a specialization of ASL without virtualization extensions (EL2=false). These two specifications are connected by 4 refinement proofs. From bottom to top these are:

- (1) **Instruction Decomposition (1).** We refactor the ASL semantics to separate the privileged state and operations from the unprivileged state. We call this refactoring S-ASL. This translation focuses purely on the ASL and we believe it to be largely parametric over the ASL execution framework (RMEM), *e.g.*, the tree-of-instructions model (ToI).
- (2) **The NOVA CPU Simulation (2).** NOVA re-organizes this operational model by pulling out the privileged state that it owns (described in the diagram as EL2) and introducing a new execution framework (RMEMn)

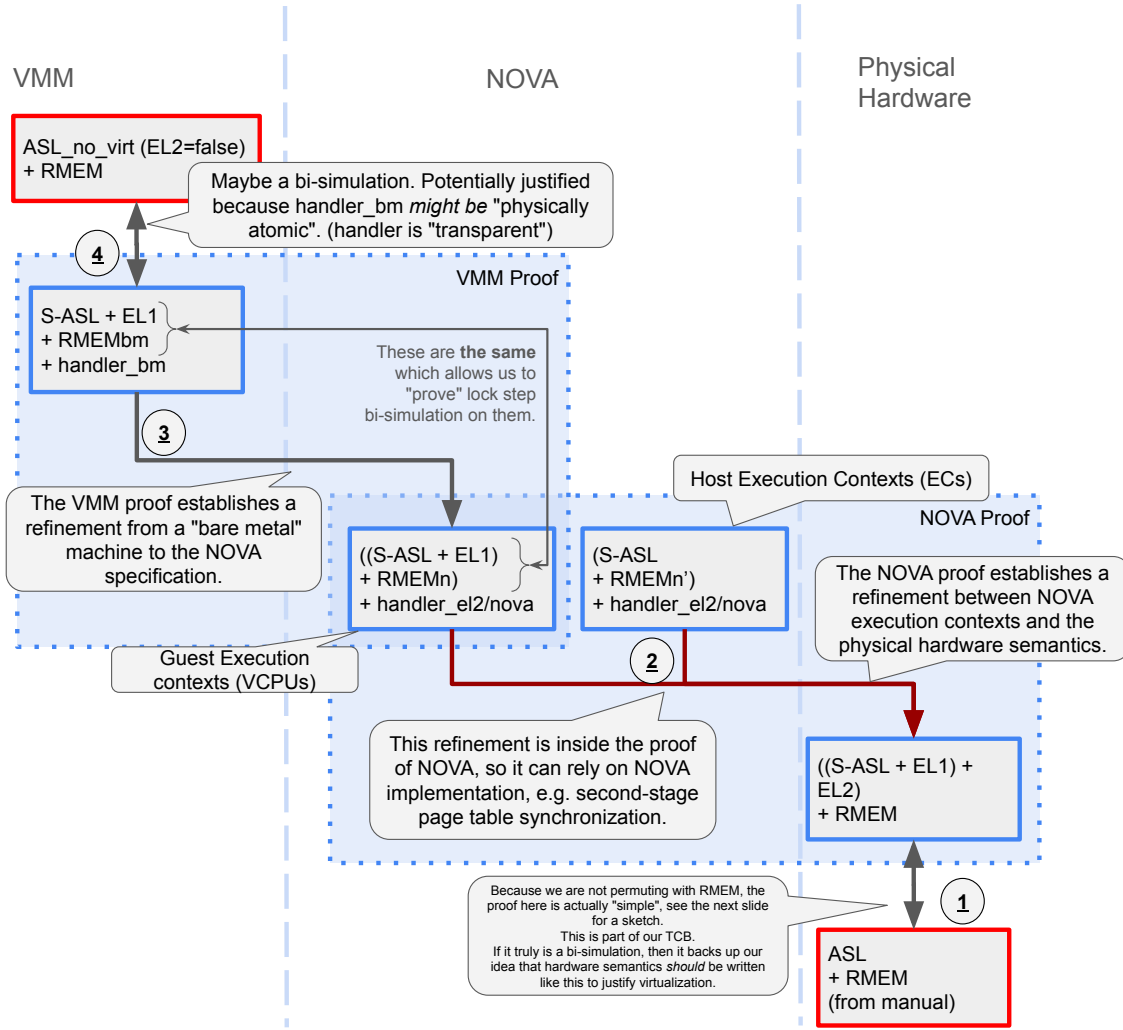


Fig. 2. Vertical decomposition of the CPU simulation.

and a handler that interprets the actions of $S-ASL+EL1$ against the NOVA model. For example, in the $S-ASL+EL1+EL2$ model, the semantics of a hypercall disables interrupts, pushes registers, and jumps to the entry in the interrupt vector table. In contrast, the NOVA handler describes the effects that the hypercall has on the NOVA state, e.g., how 2nd-stage page tables or registers are updated. Formally, this handler specification is written in a stylized subset of concurrent separation logic, but we believe that this is merely a convenience and could be written purely operationally—though doing so would be a maintenance burden.

- (3) **The VMM Simulation (3)**. VMM virtual CPUs are implemented by a combination of (a) hardware virtualization (provided by NOVA virtual CPUs) and (b) software emulation (that runs as user-space code on the host). The focus of the document is establishing these two parts of the CPU refinement.
- (a) The first part is established by hardware virtualization of unprivileged instructions, that is, those that execute internally to S-ASL+EL1 + RMEM. Here the justification the implementation component (S-ASL+EL1 + RMEM in the bottom box) refining the specification component (S-ASL+EL1 + RMEM in the top box) is a lock-step simulation proof with a simulation relation that is (almost) equality. We discuss the reasons that this simulation cannot be on-the-nose equality when we discuss the CPU simulation (§5.2.2).
 - (b) The second part is about privileged operations, that is, operations that invoke the `handler_e12` / `handler_bm` and where the implementation does not coincide with the specification. Here, `handler_e12` depends on EL2 configuration controlled by VMM, and will choose either HW execution with altered semantics, or trapping to the VMM. We prove that HW execution refines the bare-metal by showing that the configuration of the EL2 state causes the hardware to preserve the simulation relation. For instance, when reading from a given GPA, `handler_e12` traverses the 2nd-stage page tables controlled by VMM (via NOVA), and, depending on the translation result, `handler_e12` will either access implementation memory that refines the specification memory, or trap to VMM.
 - (c) Finally, we prove refinement in the case where `handler_e12` traps to VMM C++ code (§5.3). This proof is a “standard” program verification of the VMM code, which is written in C++ and verified using our BRiCk semantics. The use of separation logic in both the NOVA interface and BRiCk makes it natural to connect the C++ code verification to the rest of the verification.
- (4) The connection between S-ASL + RMEMbm and the top-level ASL semantics (4) is similar to the refinement relationship established in (1), with the exception that the refinement is flipped, *i.e.*, we need that S-ASL refines the ASL, instead of the other way around. If the simulation between S-ASL and ASL is truly a bi-simulation arising from functional program equivalence, as we believe it to be, then such a proof should be straightforward.

By design, this setup yields a proof of VMM hardware virtualization that is largely parametric in the S-ASL + RMEM semantics, so the crux of the proof lies outside the VMM. In this report, we focus on (3)—the VMM simulation.

2.2 Reasoning About C++ Programs

As the BlueRock VMM is implemented in C++, we carry out our verification using the BlueRock BRiCk program logic [9]. Similar to the NOVA specification, BRiCk expresses the state of the C++ abstract machine directly using separation logic and expresses the reasoning principles for individual program fragments as predicate transformers in the weakest (liberal) pre-condition style. The uniform use of separation logic for NOVA and C++, along with the open-world model of separation logic, makes it easy to connect these formalisms together and reason about C++ programs that interact with NOVA state.

Nonetheless, formally connecting the C++ source code to the user-space semantics provided by NOVA would require a verified compiler or translation validation. While an interesting and challenging problem, we explicitly keep this out of scope and assume specifications that allow us to translate NOVA-level weakest pre-conditions into C++ level weakest pre-conditions.

2.3 Refinement in Separation Logic

We carry out our proof of the bare-metal property directly within separation logic. This choice allows us to directly work with the specification of C++ and NOVA and to leverage separation logic’s support for modular,

```

Definition AuthSet.frag {lts : LTS} : gname → set lts.(state) → PROP.

Lemma narrow (lts : LTS) : ∀ γ a b, a ⊆ b → NonEmpty b → AuthSet.frag γ a ⊢ |={}=> AuthSet.frag γ b.

Definition updater {lts : LTS} E γ : PROP :=
  ∀ (ss ss' : set lts.(state)) (STEP: ∀ s', s' ∈ ss' → ∃ s, s ∈ ss ∧ lts.(step) s None s') ,
  AuthSet.frag γ ss ⊢ |={E}>= AuthSet.frag γ ss'.

(* Obligation to update the state with a request step *)
Definition requester (lts : LTS) E γ (Ev : set lts.(Event)) (Q : lts.(Event) → PROP) : PROP :=
  AC << ∀ s, AuthSet.frag γ {[ s ]} *
    [ ∀ e, e ∈ Ev → ∃ s', lts_step lts s (Some e) s' ] >> @ T \ E, ∅
    << ∃ e, [ e ∈ Ev ] * AuthSet.frag γ {[ s' | lts_step lts s (Some e) s' ]} , COMM Q e >>.

(* Obligation to update the state with a response step *)
Definition committer (lts : LTS) E γ (Ev : set lts.(Event)) (Q : lts.(Event) → PROP) : PROP :=
  AU << ∃ ss, AuthSet.auth γ ss >> @ T, T \ E
    << ∀ s e s', [ s ∈ ss ∧ e ∈ Ev ∧ lts_step lts s (Some e) s' ] *
      AuthSet.auth γ {[ s' ]} , COMM Q e >>.

Definition transports (lts1 lts2 : LTS) E γ1 γ2 : PROP :=
  □ ∀ (Ev1 : set lts1.(Event)) (Ev2 : set lts2.(Event)) (dual : dual_evts Ev1 Ev2) Q,
    requester lts1 E γ1 Ev1 Q * committer lts2 E γ2 Ev2 (λ e2, ∃ e1, [ e1 ∈ Ev1 ∧ dual_evt e1 e2 ] * Q e1).

Lemma transport_singleton lts1 lts2 Ev1 Ev2 γ1 γ2 Q :
  dual_evts Ev1 Ev2 →
  transports lts1 lts2 γ1 γ2 ⊢
  requester lts1 E γ1 Ev1 Q *
  committer lts2 E γ2 Ev2 Q.

Theorem decompose (ltses : Name → LTS) γ :
  let wholeLTS := parallel_compose ltsets in
  ∀ ws : wholeLTS.(state),
  AuthSet.frag γ ws ⊢
  |={ T }=> ∃ γs : Name → AuthSet.gname,
    (* ghost ownership of every component decomposed *)
    [*list] c ∈ Name, AuthSet.frag (lts:=(ltsets c)) (γs c) (ws c) *

    (* Ability to make τ steps for each component on the corresponding ghost ownership *)
    [*list] c ∈ Name, updater (ltsets c) E (γs c) *

    (* Ability to make internal communication between two components *)
    [*list] (c1, c2) ∈ Name * Name, transports (ltsets c1) (ltsets c2) E (γs c1) (γs c2) *

    (* Ability to make externally visible communication for every component *)
    [*list] c ∈ Name, external_requesters (ltsets c) (γs c) *
    [*list] c ∈ Name, external_committers (ltsets c) (γs c).

```

Fig. 3. The core components of the Spectra library. Hereafter we write **Definition** ns.name as syntactic sugar for **Definition** name in namespace ns, and similarly for other commands.

open-world reasoning. We follow CaReSL-style [16] techniques exposed through BlueRock’s Spectra library [10]. Spectra provides a ghost state construction ($\text{AuthSet.frag } \gamma \text{ } ss$) to embed the state of an arbitrary label-transition system (LTS) into separation logic (Figure 3). Here, γ is a (opaque) ghost name and ss is a set of potential states of the underlying LTS. Importantly, it is *not* unknown which of the states the LTS is in, rather, it is possible for the LTS to be in any of these states without changing the visible trace up to this point. The narrow rule allows the prover to refine this set “for free”, reducing the amount of non-determinism.

Spectra also provides reasoning principles for stepping the specification state. The updater reasoning principle (Figure 3) allows updating the set ss to ss' if *all* states s' in ss' are reachable by a silent step from *some* state s in ss .

Taking non-silent steps is slightly more complex because we want component-local specifications that are independent of the communicating partner. To capture this, Spectra exposes requesters and committers (Figure 3). While standard process calculi generally do not distinguish between the request and commit side of communication, the sequential nature of ghost state update within the separation logic of Iris makes it easier to express when we distinguish these sides. These two assertions enable using the standard communication rule for parallel compositions in process calculi. The transports property connects the requesters and committers of two communicating components ($lts1$ and $lts2$) together. Intuitively, it is saying that, for a non-silent step of $lts1$ with an event in $Ev1$ that is dual (communicating) with a non-silent step of $lts2$ with an event in $Ev2$, one must first satisfy the obligation to update the state of $lts1$ and then the obligation to update the state of $lts2$, to complete the step of the whole system composition.

Spectra supports separation logic analogs of the core process calculi primitives such as parallel composition and event hiding. The main theorem we use is the decompose theorem which supports reasoning about parallel composition. Specifically, this theorem allows a user to decompose the monolithic ownership $\text{AuthSet.frag } \gamma \text{ } ss$ of a composed LTS wholeLTS into separated ghost ownership $\text{AuthSet.frag } (lts := (ltses \text{ } c)) \text{ } (\gamma \text{ } c) \text{ } (ws \text{ } c)$ for every component c in wholeLTS . Together with these ownership, one also acquires the properties needed to update the state of each component. We will see the concrete application of this theorem in Section 4.1, in order to decompose the whole-machine model described in Section 3.

Spectra provides an idiomatic specification for a program that maintains a refinement (e.g., the VMM). The idiomatic specification provides ownership of the Spectra ghost state in an initial state that is compatible with any other resources available as the pre-conditions of the program. For example, if VMM::main() is the program entry point of the VMM code, its idiomatic specification should have the following shape:

```
\pre AuthSet.frag  $\gamma_{bm}$  (bm.init_state guest_image)
\pre implementation.resources (* e.g. ownership of the function arguments *)
VMM::main()
```

That is, the program takes as inputs the physical resources it needs for the implementation, as well as the bare-metal ghost state. Proving this idiomatic specification of VMM::main() means that we have to establish and maintain the refinement of the bare-metal by the implementation.

2.3.1 Formally Proving the Refinement. Ultimately, our aim is to prove a refinement between operational models. Formally, the fully-operational characterization of the bare-metal property would be the following.

```
 $\forall$  guest_image tr final_state,
  lts_step_star physical.lts (physical.lts.(init_state) guest_image) tr final_state  $\rightarrow$ 
 $\exists$  tr' final_state',
  lts_step_star bm.lts (bm.lts.(init_state) guest_image) tr' final_state'  $\wedge$ 
  erase_tau tr = erase_tau tr'.
```

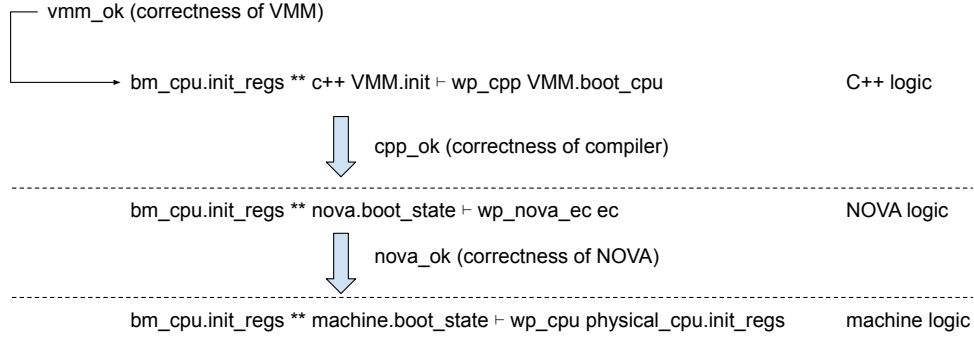


Fig. 4. Vertical decomposition of the end-to-end refinement.

Here, $\text{bm.lts.}(\text{init_state})\text{guest_image}$ is the initial state of the specification machine, *i.e.*, the bare-metal machine that is running the guest code with no hypervisor. The $\text{physical.lts.}(\text{init_state})\text{guest_image}$ is the initial state of the full hypervisor stack including NOVA, the VMM, and other ULTRA platform applications such as the master controller, with a startup image that correctly configures the stack to run the VMM with the given image. lts_step_star is the transitive closure of the step relation; tr is the sequence of event labels, and erase_tau filters silent τ steps, leaving only labels of external events, such as network messages exchanged with the outside world. Crucially, $\text{erase_tau tr} = \text{erase_tau tr'}$ asserts that the labels of *external* events coincide in the implementation and the specification machine. So, if a guest running under VMM exchanges certain network packets or performs other kinds of external I/O, there is a step sequence for the matching bare-metal guest that exchanges the same packets or doing the same external I/O.

In the trace-based simulation property mentioned above, we do *not* prove that final_state and final_state' are related: our proof maintains a stronger internal simulation between them, but as we will see, the details of the simulation can be complicated. Hence, we only use the internal simulation as a proof device, and we do not expose it in our top-level statement.

2.3.2 A Sketch of the End-to-End Refinement Proof. We note that, in this report, we focus exclusively on the VMM source code, excluding aspects such as compiler correctness and the correctness of NOVA and other components. While these aspects are also important to a final foundational proof, strictly starting at the bottom would prevent us from being able to explore the techniques that we explore in this document and would have precluded us from finding the numerous bugs that we found in the course of the verification.

Figure 4 sketches how we could connect the proof of our VMM to the proof of the other components in the system to build a closed (up to explicit assumptions) proof of the refinement via adequacy. Here, the top-most entailment (vmm_ok) is what we are focusing on. It establishes the soundness of the VMM source code against the BRiCK C++ logic. The result of this step is the proof of the assertion wp_cpp for the weakest pre-condition of a C++ thread running VMM code.

The correctness cpp_ok of the C++ compiler (when the object code is running against NOVA) translates wp_cpp and the initial state of the C++ program, into an assertion wp_nova_ec for the weakest pre-condition of the NOVA execution context running the application. The BlueRock ULTRA platform, when configured appropriately, has a specification that establishes this pre-condition by running the master controller which will set up resources, load the VMM binary, set up its stack, and jump to the entry point.

wp_nova_ec is then discharged by the correctness of NOVA, nova_ok , to get us to wp_cpu , the weakest pre-condition of the physical CPU execution. That is, when correctly configured, NOVA will start in a state from the

boot registers, set up some state, and then begin running the startup application, which is the ULTRA master controller.

Finally, `wp_cpu` is discharged by the soundness of the logic against an operational specification of the machine in order to extract a final, end-to-end operational refinement of the full system.

3 A WHOLE-MACHINE VIRTUALIZATION SPECIFICATION

We consider a natural top-level specification of the VMM correctness as a refinement property: **the VMM running a guest system refines the behavior of a bare-metal computer without virtualization extensions running the same guest system**. We call this property the bare-metal propertyTM. The property is useful in various ways:

- It provides confidence that workloads running on physical machines can safely be migrated to run on virtual machines without introducing exotic behaviors not permitted by the hardware specification.¹
- Consequently, the property can be linked with verifications that use the bare-metal model semantics, for example, verifications of guest code. In other words, guest verification can be performed while being agnostic over virtualization extensions, and verification results can be combined with the bare-metal property to derive end-to-end results over a virtualized system.
- It guarantees the integrity of the guest. Thinking about integrity as semantic invariants about the guest system, conformance to this rich functional specification allows us to prove that any property preserved by any arbitrary behavior of the hardware architecture is also preserved when running under virtualization.
- It guarantees some level of confidentiality up to the non-determinism of the underlying hardware architecture and side channels. Here, functional correctness ensures that architectural visible state conforms to the specifications allowed by the hardware, but the specification is too weak to infer the absence of channels through explicit non-determinism, *e.g.*, the value of an unspecified flag register could be used to communicate information. While we believe that controlling these channels is important, precise characterizations of these properties on mainstream architectures still seems out of reach.

To state the bare-metal property formally, we need to define the semantic model of a bare-metal computer.

3.1 A Whole-Machine Model

Drawing on inspiration from the systems community [5, 11, 12] and hardware architectural specifications [3], our computer model is built around the central concept that computers are distributed systems. The components of a computer include CPUs, memory, interrupt and bus controllers, and (peripheral) devices. These components interact through explicit messages and can experience network effects such as delay and re-ordering, though we will not consider other forms of noise such as message loss. Beyond being easily understandable to systems engineers, this distributed-system model has a number of benefits:

- It enables us to build the full machine model compositionally, incrementally, and in an open-ended way. It also gives us freedom to model different components at different levels of precision.
- It makes communication between components explicit and makes precise that local steps are truly local. In other words, a component's external communications clearly define its interfaces.
- Finally, it allows us to decompose the overall implementation and verification along similar lines.

3.2 Communications in the Distributed Model

In this distributed model, components are connected through various communication channels, the most important of which is the core “system bus”, visualized in Figure 5. In this model, input-output (IO) communications with the environment are only possible through devices.

¹We acknowledge that software written against a particular hardware implementation may exhibit bugs when run on a correct VMM because the VMM may make different implementation choices that are permitted by the architecture. This is largely unavoidable in practice.

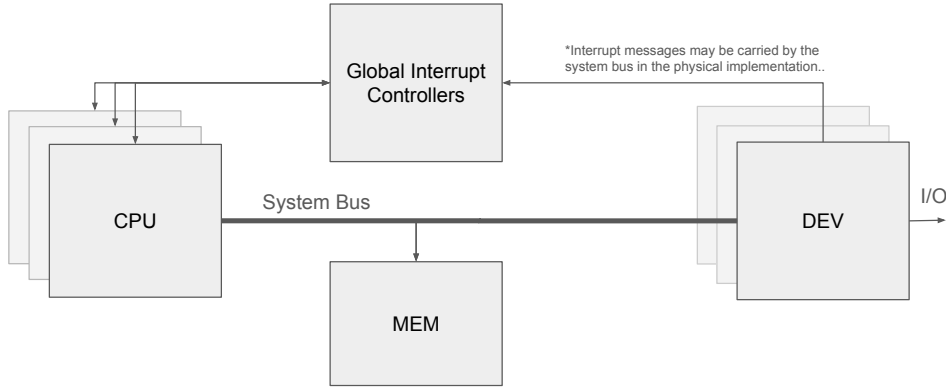


Fig. 5. Machine components connected by a system bus.

```

Variant Space := Memory | PIO | Config | Message.
Variant Role := Requester | Completer.

Variant Event :=
| Read(r : Role) (sp : Space) (pa : paddr) (bytes_sz : N) (offset : N) (rs : ReadState)
| Write(r : Role) (sp : Space) (pa : paddr) (bytes_sz : N) (offset : N) (bits : N) (ws : WriteState).

```

Fig. 6. Universal Event type for communication.

There are several interesting aspects about the modeling of the communication channels.

3.2.1 Message Types and Channel Types. There are various types of communications that can happen in the system. A CPU can access main memory, can access its own registers or other registers shared with other CPUs, can program registers of devices, can have I/O access to devices, and can send interrupts to other CPUs. A device can access its own registers as well as registers of other devices, can access main memory, and can also send interrupts to CPUs. Traditionally these communications happen in their own channels: there are channels for main memory accesses, for register accesses, and for raising interrupts.

While still supporting legacy access channels, modern systems introduce a unified interface through memory accesses: registers and I/O ports can be mapped into some reserved memory address range (MMIO—memory-mapped IO); and interrupts can be raised by writing to some reserved memory address range (MSI—message-signaled interrupt).

To support different types of communications, in the formal model of the specification, we introduce a common event type that abstracts and covers all message/channel types, in Figure 6. An Event can be a read or a write and occurs in space which is one of Memory, PIO, Config, or Message. The spaces are inspired by the PCI standard. Memory space is used for all memory accesses, including MMIO and MSI. PIO is used for legacy IO messages. Config is mainly used to program registers. Message can be used for other types of communication, including raising interrupts. The accessed address, size, and offset identify the target of the access, and can be interpreted depending on the space. All components in the system then communicate using the common Event type. In fact, every component is wrapped in a “bus interface” that acts as an adapter to translate different types of events into the common type.

3.2.2 Asynchronous Communication vs. Synchronous Communication. Different spaces in the Event type have different communication behaviors. One such difference is whether the communication takes effect immediately, or requires more time and further communication to complete the effects. The former is called synchronous communication and the latter asynchronous. The Event type (Figure 6) is constructed so that it can be used to model both types of communications.

Synchronous communication is typically used to model for accesses to main memory. For example, a read or a write can be modeled as having effects immediately on the main memory. Asynchronous communication is typically used to model configuration or IO accesses. An asynchronous communication involves two events: one event (the initiation event) communicates the request from the Requester to the Completer of the request, and the other (the completion event) communicates the response from the Completer to the Requester. The two events are linked by a shared transaction identifier.

The choice to model a communication as synchronous or asynchronous depends on the logical atomicity of the effects within the context of the whole system. Generally, asynchronous communications are easier to justify, but are more difficult to reason about since they can be interleaved with other events and often require maintaining state between the messages.

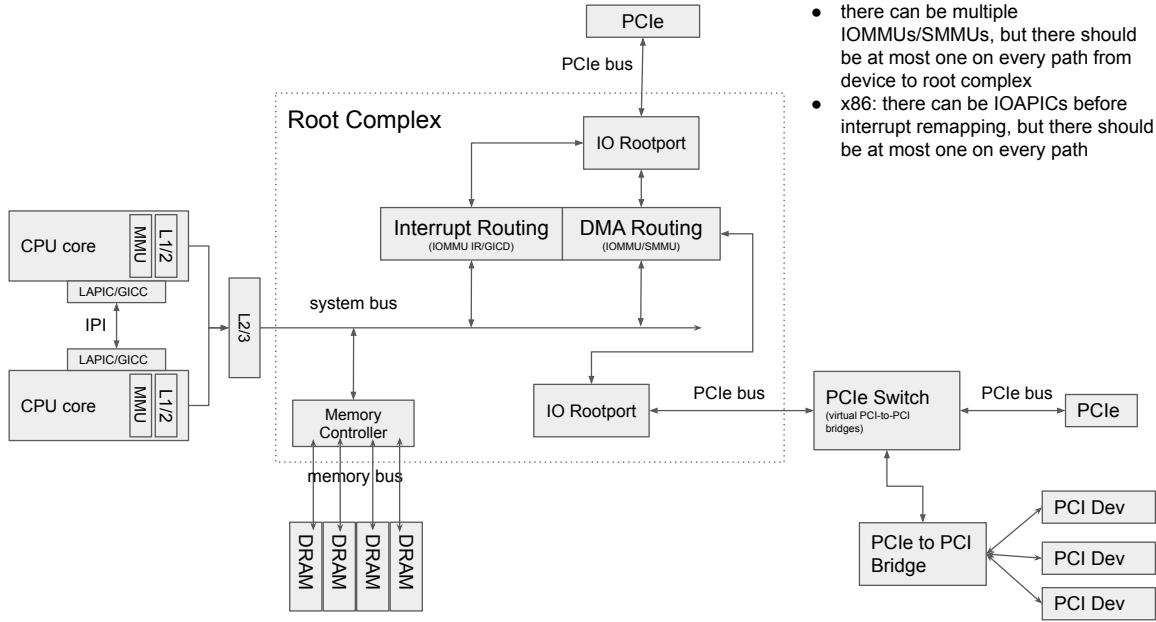
3.2.3 Hierarchical Topology. Modern computers are wired together in a rich topology. There can be multiple buses connecting CPUs themselves; CPUs and main memory are connected to the main system bus; devices can be laid out on multiple buses before being connected to the main system bus. Furthermore, there can be IOMMUs/SMMUs and interrupt remapping units that sit between the devices and the main system (also called the root complex) and that filter or redirect DMAs and interrupts from the devices. Figure 7a illustrates a simple topology.

In modeling the whole system, we abstract away from the details of the topology of how components are connected together. Instead, we consider a flat model where components can almost always communicate directly to one another. To model the filtering and redirecting effects of IOMMUs/SMMUs and interrupt remapping units, we use asynchronous communications and require that communications between devices and the root complex are relayed by the IOMMUs/SMMUs/interrupt remapping units. Such a model is illustrated in Figure 7b.

3.2.4 Static vs. Dynamic Topology. Another important feature that we need to consider in the bare-metal model is that the topology of devices outside of the Root Complex can be dynamic. This means that devices can be mapped and unmapped into the system address spaces while the machine is running. In addition, this sort re-configuration is completely controlled by the software running on the machine, which makes it difficult to rule out pathological topologies.

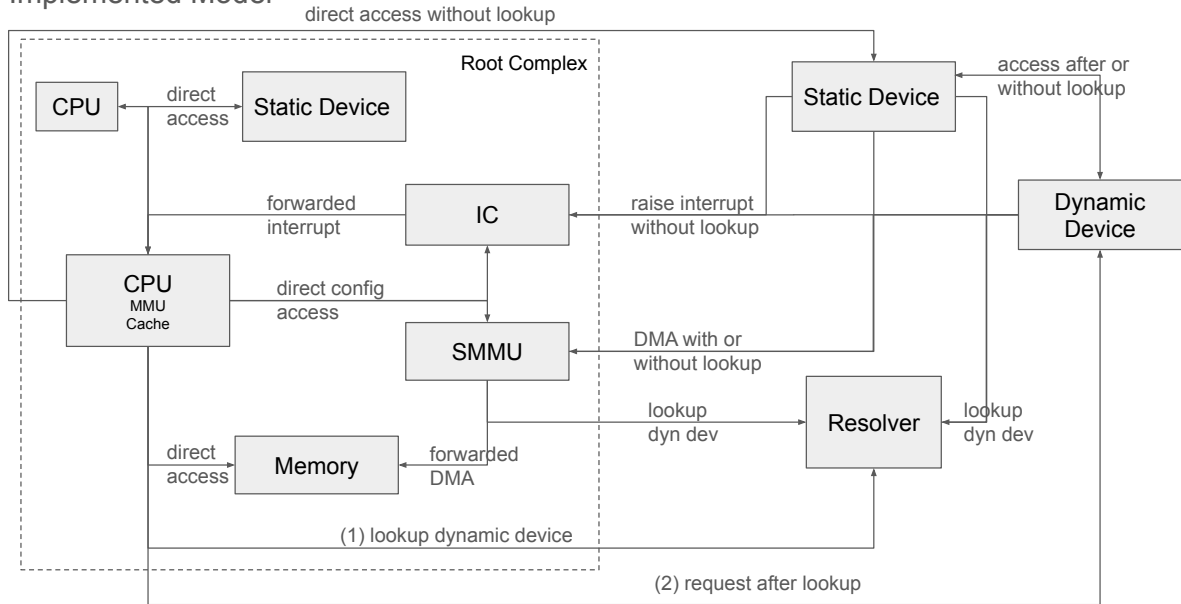
In practice, the mappings of devices can be very complex. For example, PCI devices can have their MMIO registers moved in the address space, and their IDs can also be changed by changing PCI switches. Communications to and from these devices are in the form of messages that can be routed through the topology, and some messages can be delivered before other messages (reordering).

We abstract away many of these details by introducing a component called the Resolver (see Figure 7b) that takes care of the lookup of a target of an event (abstracting the routing), and of the update of such mappings of addresses to devices. If a communication is known to only involve the static part of the topology, the communication can happen directly between the two components, and either asynchronous mode or synchronous mode can be used. If the communication targets a component in the dynamic topology, the communication needs to go through the Resolver first to look up the target. Asynchronous communication mode must be used in this case.



(a) An example topology of a physical machine.

Implemented Model



(b) The implemented model of Figure 7a. The Resolver centralizes and abstracts the routing of messages.

Fig. 7. Modeling of the whole physical machine.

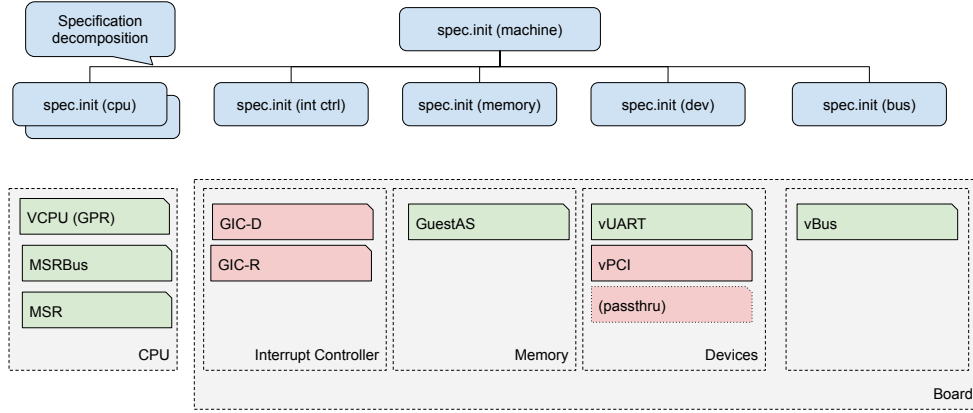


Fig. 8. The monolithic guest specification is decomposed into separate ownership that aligns with the architecture of the VMM implementation.

4 THE ARCHITECTURE OF THE BLUEROCK VMM

The VMM is responsible for the virtual machine refinement through the ghost ownership of the bare-metal model presented in [Section 3](#). However, this monolithic model is not directly suitable for a modular verification. The bare-metal ghost ownership needs to be decomposed to reflect the separation of responsibilities within the VMM. [Figure 8](#) shows the decomposition of the specification state and how it is divided among the static modules in the VMM.

Each of the major components of the system machine model, e.g. CPU, memory, devices, etc, is implemented by a separate module within the VMM implementation. Each module maintains its own state, with its associated invariants, and exposes APIs that reflect communication between components in the whole machine model. In this section, we cover the high-level architecture of the BlueRock VMM and describe how we decompose the specification ghost state ownership and divide it amongst the VMM’s modules. Starting from [Section 5](#) we will discuss the dynamic behavior of each component, how communication between components is reflected in the specification, and how the simulation relations between the specification and implementation components are maintained.

4.1 Decomposing the Monolithic Specification

We leverage invariants and fictional separation to decompose the monolithic specification state into disjoint specification state that we can use in the simulation relation for each component. The decomposition lemma, given in [Figure 9](#), is the direct application of the decomposition theorem from the Spectra library (see [Section 2.3](#)) on the whole machine model given in [Section 3.1](#). In essence, the lemma consumes ownership of the monolithic model and produces separate ownership for each component in the model. Each of these ghost ownership will be used to construct and maintain the simulation of its corresponding component. The remaining conjuncts (e.g., updater, transports, external_requesters, and external_committers) characterize the ability to update the whole machine ghost state in terms of updating the per-component ghost state, effectively reflect into separation logic the communication patterns among components and between components and the external world.

```

Lemma decompose_bm_machine :
  AuthSet.frag  $\gamma_{bm}$  bm.machine.(init_state) ⊢
  |={T}=>
    let Name := bm.cpus ∪ {[ MEM , RESOLVER ]} ∪ bm.devices in
    ∃  $\gamma$ : Name → AuthSet.gname,
    (* ghost ownership of CPUs *)
    [*list] c ∈ bm.cpus, AuthSet.frag ( $\gamma$  c) (bmCPU.lts c).( init_state) *
    (* ghost ownership of main memory (DRAM) *)
    AuthSet.frag ( $\gamma$  MEM) DRAM.(init_state) *
    (* ghost ownership of the Resolver *)
    AuthSet.frag ( $\gamma$  RESOLVER) RESOLVER.(init_state) *
    (* ghost ownership of devices, including interrupt controllers *)
    [*list] d ∈ bm.devices, AuthSet.frag ( $\gamma$  d) (bm.dev.lts d).( init_state) *

    (* Ability to make  $\tau$  steps for each component on the corresponding ghost ownership *)
    [*list] c ∈ Name, updater (bm.lts c) ( $\gamma$  c) *
    (* Ability to make internal communication between two components *)
    [*list] (c1, c2) ∈ Name * Name, transports (bm.lts c1) (bm.lts c2) ( $\gamma$  c1) ( $\gamma$  c2) *
    (* Ability to make externally visible communication for every component *)
    [*list] c ∈ Name, external_requesters (bm.lts c) ( $\gamma$  c) *
    [*list] c ∈ Name, external_committers (bm.lts c) ( $\gamma$  c)

```

Fig. 9. The decomposition of the bare-metal whole-machine ghost state into ghost state of its components.

4.2 The VMM Components

Figure 8 shows how the modules of the BlueRock VMM mirror the structure of the machine model.

CPU. Each CPU component owns the specification state for the full bare-metal CPU including its general purpose registers, its system registers (ARM) or model-specific registers (MSRs) (x86), as well as the local interrupt controller (GICC in ARM, LAPIC in x86) state. Unlike the other components in the system, the state of the CPU is not affected concurrently by the rest of the system with the exception of receiving interrupts from the interrupt controller. Nevertheless, the relationship between the specification state and the implementation state, which comprises both NOVA system resources and C++ resources, is quite dynamic throughout the lifetime of the CPU. We discuss this relation in Section 5.

Memory. The VMM’s memory component emulates the memory of the specification system. Memory can be accessed through hardware virtualization as well as through software emulation concurrently, from both CPUs and devices. This makes the memory component a highly concurrent, and performance critical component. Good performance here relies on hardware virtualization technologies, such as 2nd-stage page tables, that are encapsulated by NOVA. However, the resources must also be accessible by software emulation. We discuss the way that we address supporting this range of clients in Section 6.

The Bus. The hardware bus provides the routing logic for messages between components. For example, “memory” accesses issued by a CPU may, in fact, be MMIO accesses that communicate with registers on devices. Simple buses, such as those that exist in embedded systems, are often static and are relatively easy to model and verify. More complex buses, such as PCI buses, allow for dynamically relocating devices and adding and removing devices at runtime (hot-plugging and hot-unplugging). We discuss the support for buses in Section 7.

Devices. The BlueRock VMM supports a range of virtual devices that are implemented in software and can connect to the rest of the ULTRA platform. For example, a virtual serial device forwards characters to a (separately

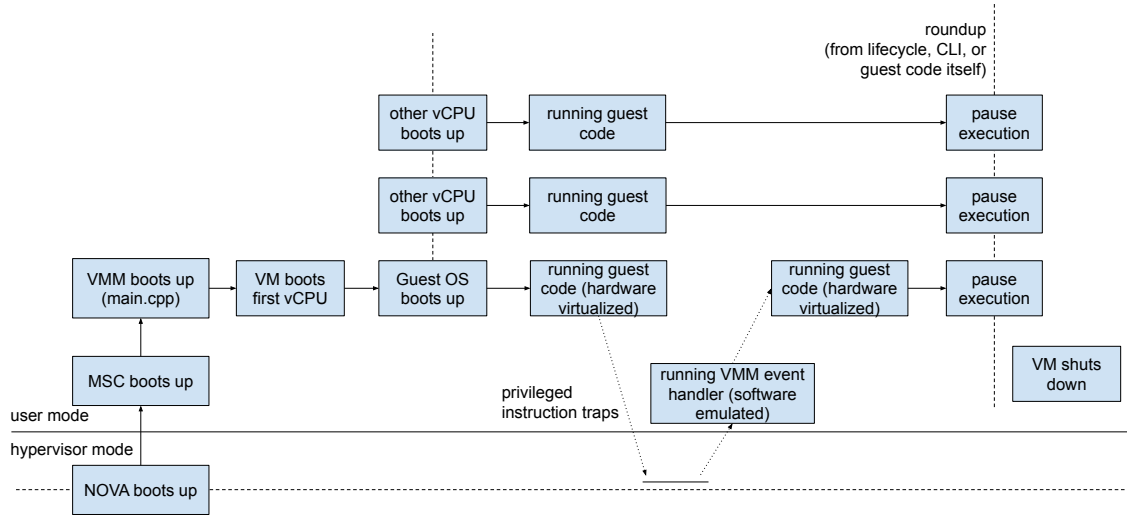


Fig. 10. An example life-cycle of a guest virtual machine running atop the BlueRock VMM.

verified) console multiplexer, and a VIRTIO-Net device (implemented in another process) provides a virtual network switch to clients running within ULTRA. The architecture of the VMM makes this set of devices extensible by providing a uniform interface to implement device emulators. We discuss this interface in [Section 8](#).

The BlueRock VMM also supports “passthrough” devices, which are devices in the physical implementation system that are directly accessible by the guest. While the implementation of these devices are completely different from that of virtual devices, we believe that our system architecture is general enough to support them.

Interrupt Controller. One special device within the system is the interrupt controller. Abstractly, an interrupt controller is simply a device like any other: it responds to MMIO, MSR, and interrupt events, and raises interrupts. We distinguish it from other devices because it operates as a distributor of interrupts and modern hardware provides several features to make hardware virtualization of interrupt controllers more efficient. While we have not yet verified an interrupt controller implementation, our architecture does account for them, and we believe that it is tractable to verify one against the specifications. The main technical difficulty that we have identified in accurately implementing interrupt controllers is that their specifications leverage the ability to access large quantities of data atomically. For example, an operation from the interrupt controller might find the highest priority interrupt that is enabled out of potentially a thousand interrupts with 8-bit priority settings where both the enabledness and the priority of any given interrupt can be changed concurrently by multiple actors within the system.

4.3 Life-cycle of a Guest Virtual Machine

Dynamically, the VMM provides implementation for initializing its components during the boot up of the VM, in `VMM::main()`. When being booted up by NOVA, `VMM::main()` will:

- set up system resources so that the guest (the VM) can be hardware-virtualized (through NOVA), and
- set up event handlers so that faults during hardware-virtualized execution of the guest are handled by VMM code (software-emulated), and

- set up interfaces so that the guest execution can be managed externally (e.g. for monitoring or introspection). The lifecycle component provides such a programming interface, and a command-line interface is also available.

Furthermore, the implementation includes a roundup mechanism (`vcpu_roundup`) to stop the execution of vCPUs. This mechanism can be used to restart or shutdown the VM, or to pause the VM for introspection. More details on the roundup process are given in [Appendix A](#).

Example Life-Cycle of a guest virtual machine running on VMM. [Figure 10](#) illustrates an example life-cycle of a guest OS running on a VM on top of VMM.

After booting up, NOVA will boot the BlueRock root task—the master controller (MSC) that provides the runtime environment for user applications. Among these applications are a VMM (there can be multiple VMM instances, one per VM) that will boot up next.

As the VMM boots up, it sets up the guest address space, maps the guest OS image, sets up the virtual bus, various virtual devices, including interrupt controllers, and ties them to the board and the virtual CPUs. The boot process does this using a user-provided configuration of the VM. After the vCPUs have finished running some VMM startup code and have become idle, the VMM starts the first vCPU (bootstrap vCPU) that will boot the guest OS. Guest code will now start and use other vCPUs according to the guest needs. We discuss the verification of the startup process in [Section 9](#).

Most of the time guest code will run on hardware virtualization (properly set up by NOVA and the VMM). However, if guest code executes an instruction that accesses privileged state, or an instruction that accesses memory with insufficient permissions, or an instruction that accesses a virtual device that is purely implemented in software, the hardware will trap into NOVA, which then delegates the handling to a corresponding VMM event handler. The VMM event handler code is run to emulate the guest instruction (software emulation). Afterwards, guest code can resume running on hardware virtualization. In our refinement proof, we have to maintain the simulation of the bare-metal guest behaviors during both hardware virtualization (using the semantic model of hardware virtualization) and software emulation (using the correctness of VMM C++ code). We discuss the formal specification and verification of hardware virtualization and software emulation in [Sections 5 to 8](#).

During the execution, a roundup request may be issued, from the lifecycle module, or from the command-line interface, or from the guest code itself (through virtual firmware). A completed roundup will stop the execution of all vCPUs, after which the VM may resume, restart or shutdown.

5 THE CPU SIMULATION

The core functionality of the VMM revolves around the CPU component. In this report, we use the term CPU to refer to a single hardware thread within a CPU. CPUs are the main drivers of the rest of the system and support relatively few (spontaneous) incoming messages.² This means that maintaining the simulation relation of a single CPU is largely a sequential task that is separate from the simulation relation of other system components.

We begin with an overview of the state associated with virtual CPUs ([Section 5.1](#)), which contains both NOVA as well as C++ state declared and manipulated directly by the VMM implementation. This state supports interesting functionalities beyond normal control, such as pausing all CPUs (roundup, see [appendix A](#)) as well as system reset. In the rest of the section, we discuss the various pieces of the simulation proof for CPUs connecting it back to the semantics of virtual CPUs exposed by NOVA ([Figure 12](#)). Specifically, we cover silent steps ([§5.2.1](#)), faulting steps ([§5.2.2](#)), trapping steps ([§5.3](#)), and system register accesses ([§5.4](#), [§5.5](#)). We leave memory steps for [section 6](#).

²Interrupts are the notable exception, but we will ignore those in this section.

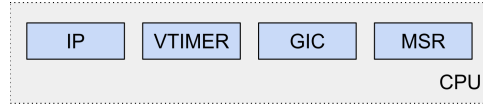


Fig. 11. Components of CPU model: in the implementation CPU model, system registers are separated from the IP and put into the MSR component. Meanwhile, the specification (bare-metal) model does not have the MSR component, and still keeps system registers in the IP.

5.1 The CPU State & Behavior

The state of the CPU is maintained in both NOVA and in the VMM code, and the relationship between different pieces of the state changes throughout the execution of the VMM.

5.1.1 CPU Components. While we often think of the CPU as a single hardware execution unit, modeling the state as a collection of separate components allows us to modularize reasoning on CPUs. We use the decomposition of the CPU model shown in Figure 11.

- An **instruction pipeline** (IP) who drives the execution of instructions—it tracks the state of the register file and the local state of currently executing (possibly due to speculation) instructions.
- A **local interrupt controller** (GIC) who receives interrupts from the global interrupt controller and buffers them at the CPU.
- The **timer** (VTIMER) who provides timer interrupts through the local interrupt controller.
- The **system/model-specific registers** (MSR) who provides access to systems components such as performance counters or CPU Feature Registers.

In the hardware architectural specifications [3], system registers together with general-purpose registers belong to the instruction pipeline. However, system registers have special interactions with virtualization, in that accesses to them are mediated by the control registers, and they can trap. Therefore, we follow the decomposition of the CPU semantics as presented in Section 2.1.1, and model MSRs as their own component, leaving the IP with general-purpose registers.

The bare-metal bmCPU.lts vs. the implementation vCPU.lts . Formally, the label-transition system (LTS) of the bare-metal CPU— bmCPU.lts —is the parallel composition of the LTSes of its components. This should be the same case for the LTS of the implementation CPU— vCPU.lts . (We also call this the virtual CPU, or vCPU for short.) However, as we shall see, the NOVA specification separates the MSR LTS from vCPU.lts , to make it easier to expose the independence of certain steps concerning system registers. Consequently, the MSR state is not part of the vCPU.lts state. We come back to MSRs in Sections 5.4 and 5.5. In Sections 5.2 and 5.3, we generally focus the simulation of vCPU.lts , and in particular, the simulation of general-purpose registers in the IP.

5.1.2 The NOVA vCPU State. The VMM implements the state of the CPU sub-components using NOVA’s support for virtual CPU (vCPU) execution contexts (ECs). The NOVA specification exposes the following separation logic ownership for vCPU state. The ghost identifier γ_{ec} uniquely identifies the EC of the vCPU.

- $ec.\text{regs } \gamma_{ec} \text{ regs}$ is the ownership of the *unprivileged* CPU state in the IP. The name *regs* is slightly misleading since the data contains much more than the architecture-visible register file, but the term leads to a good intuition about its use. *regs* excludes state for the MSR block.
- $ec.\text{ctrl_regs } \gamma_{ec} \text{ ctrl_regs}$ is the ownership of the virtual CPU’s control registers that have no analogue in a bare-metal CPU and are programmable by the VMM. For example, HCR_EL2 controls whether the hypervisor intercepts kernel-level traps. The way that the values of this register are interpreted are reflected in the handler portion of wp_nova_ec (Figure 12).

```

1 Lemma wp_vcpu_intro ec :
2   |={ T, ↑nova_ns }=> ▷ ∃ (vRegs : regs.t guest), ec.regs guest ec 1 vRegs *
3   ∀ vRegs' (oevt : option(event.t is_guest)),
4   [ lts_step vCPU.lts vRegs oevt vRegs' ] → (* CPU step *)
5   ec.regs is_guest ec 1 vRegs' → (* updated CPU state *)
6   match oevt with
7   | None ⇒ |={ ↑nova_ns, T }=> wp_nova_ec ec (* §5.2.1 *)
8   | Some evt ⇒
9     match evt with
10    | evt.Trap trap ⇒ handle_trap ec trap vRegs' (* §5.3 *)
11    | evt.Msr msr ⇒ handle_msr ec msr (* §5.4, §5.5 *)
12    | evt.Mem mem ⇒ handle_mem ec mem (* §6 *)
13    | ...
14    end
15  end
16 ⊢ wp_nova_ec ec.

```

Fig. 12. The (simplified) introduction principle for `wp_nova_ec` provides a way to reason about the behavior of a vCPU in terms of the CPU semantics (`lts_step vCPU.lts`) and a “handler” that interprets the events generated by the pipeline using NOVA state exposed through separation logic ownership.

- `ec.trappable_msrs γec tsrRegs` is the ownership of the trappable system registers in the MSR component, which NOVA separates from general-purpose registers in the vCPU, as discussed in [Section 5.1.1](#).

To allow the VMM to control guest executions, NOVA exposes the contents of many of these registers (but not all) to the VMM event handlers: on a VM exit, register values are stored to a block of memory (*User Thread Control Block*, UTCB), and updated values can be written back on VM re-entry. In particular, altering `HCR_EL2` lets the VMM control and introspect the execution of the guest without the guest’s involvement.

Beyond the architectural CPU state, NOVA also provides a *recall* mechanism [[14](#), 5.4.2], exposed through `ec.recall γec recall_flag`. When the `recall_flag` is set, NOVA will pause the execution of the execution context and invoke the recall portal. The VMM uses this functionality to inject interrupts into the vCPU and to support some life-cycle operations such as vCPU roundup ([Appendix A](#)).

5.1.3 NOVA vCPU Execution. The dynamic semantics of virtual CPUs in NOVA is captured by `wp_nova_ec`—the *weakest pre-condition* of running a NOVA execution context. [Figure 12](#) gives the general structure of `wp_vcpu_intro`. We will present more details of the definition as we present the CPU simulation proof.

`wp_nova_ec` is expressed as a per-step assertion (the `match`) over the vCPU semantics (`lts_step vCPU.lts`). Each step of the vCPU drives the execution of `wp_nova_ec` and updates the vCPU ownership. If the step is an internal CPU step, the vCPU continues at the top. If the step is a communicating step, *i.e.*, it is labeled with `Some evt`, the behavior is specified by accessing the other components of the NOVA separation logic state. For example, a vCPU fault emits a `evt.Trap trap` event and the behaviors of the rest of the vCPU execution are constrained by the `handle_trap` predicate. `handle_trap` describes, in weakest pre-condition style, how NOVA blocks on the fault handler’s local EC, copies the required state of the vCPU to the local EC’s UTCB, and begins running the local EC. We will discuss our reasoning principles around fault handlers in more detail in [Section 5.3](#). For now, we note that this behavior is still sequential, despite the fact that the handler will run on a different execution context.

5.1.4 General Proof Structure for `wp_nova_ec`. To prove that we can start vCPU execution contexts, we must prove they can run safely, *i.e.*, they adhere to the specification of the bare metal. Formally, we must prove the separation logic entailment `cpu_resources ⊢ wp_nova_ec ec` where `cpu_resources` is the resources that will be

owned by the EC thread. We note that the proof of wp_nova_ec using wp_vcpu_intro effectively corresponds to performing an induction on the vCPU’s execution trace, and for every step in the trace (every step of the induction), we must satisfy the requirements enforced by the NOVA specification for that step. Each vCPU step is *logically atomic*, which is crucial when these steps manipulate concurrent resources such as memory contents (Section 6). Formally, state changes happen between “opening” fancy updates ($\models \{\top, \uparrow \text{nova_ns}\} \Rightarrow$) and “closing” ones ($\models \{\uparrow \text{nova_ns}, \top\} \Rightarrow$, in line 7 and in the branches). Establishing this logical atomicity is a proof obligation of NOVA and is justified, in large part, by the physical atomicity guarantees of the underlying architecture.

Fortunately, all interaction between the CPU and the rest of the computer, with the exception of interrupts, is initiated by the CPU. This allows the vCPU thread to hold *exclusive* ownership over all of the CPU implementation and specification state. Effectively, steps within the CPU can be verified in a completely sequential manner. Nonetheless, we note that the CPU step relation is highly non-deterministic. This non-determinism would normally make reasoning about code running on the vCPU difficult.³ However, the bare-metal specification that we are aiming to prove provides the same degree of non-determinism, allowing us to effectively “pass the buck” up to the guest code.

5.2 The Basics of the CPU Simulation

We simulate a bare-metal CPU by combining hardware virtualization and software emulation. More specifically, we maintain a simulation between the specification bare-metal CPU state (decomposed out of the bare-metal whole-machine state—see Section 4.1) and the implementation’s vCPU state (Section 5.1), at every step of the implementation’s vCPU as described by wp_nova_ec . In this section, we discuss how to maintain this simulation.

5.2.1 The Lock-Step Simulation of CPU’s Registers, with τ -steps. Since we are using a virtual CPU to implement the corresponding specification CPU, we set up our CPU simulation such that the specification’s register file bmRegs (bm for bare-metal) is related to the implementation’s register file vRegs of the execution context. Formally, as the first attempt, this can be captured by the simple *lock-step* simulation, given in Figure 13a. Here, $\text{bmCPU.core } c \text{ } \text{bmRegs}$ is the ownership of the bare-metal bmCPU . Its numbered c with the current state bmRegs , defined using Spectra; and $\text{ec.reg } \text{guest } \text{ec } 1 \text{ } \text{vRegs}$ is that of the implementation state for the EC ec , and $\text{CPU.related_full_states } \text{bmRegs } \text{vRegs}$ is a pure simulation relation between the two states. As discussed, bmRegs contains MSR state, but vRegs does not (Section 5.1.1), and $\text{CPU.related_full_states}$ leaves MSR state unconstrained. The simulation for system registers will be discussed in Section 5.4.

Preserving the simulation for τ steps of the IP. Due to the way that the vCPU semantics vCPU.lts is constructed and exposed by NOVA, the correctness of the IP’s silent steps via hardware virtualization depends on an axiom called IP.bm_reg_step given in Figure 13b, which we commonly refer to as “hardware virtualization works for silent steps”. Here, given an *implementation* vCPU’s IP step $\text{lts_step } \text{vCPU.IP.lts } \text{vIpRegs } \text{None } \text{vIpRegs}'$, from an IP state vIpRegs that simulates bmIpRegs , we can obtain a *specification* bare-metal IP state $\text{bmIpRegs}'$ and a bare-metal IP step $\text{lts_step } (\text{bmCPU.IP.lts } c) \text{ } \text{bmIpRegs } \text{None } \text{bmIpRegs}'$, such that $\text{bmIpRegs}'$ maintains the simulation with $\text{vIpRegs}'$.

More intuitively, running an unprivileged operation under hardware virtualization (in the implementation IP) has the same behavior as running the corresponding step without virtualization enabled (in the specification IP). For example, we expect that running an addition with virtualization will compute the same value as when running it without. On the other hand, memory accesses that pertain to the addition (such as fetching the instruction, or reading operands and storing results) have more complex semantics under virtualization, so we model them as non- τ memory steps (whose simulation is discussed in Section 6).

³The difficulty of this reasoning is the heart of weak memory reasoning.

```

Definition CPU.related_full_states bmRegs vRegs : Prop :=
  IP.related_full_states bmRegs.(ip_state) vRegs.(hw_ip_state) ∧
  GIC.related_full_states bmRegs.(gicc_state) vRegs.(gicv_state) ∧
  TMR.related_full_states bmRegs.(tmr_state) vRegs.(vtimer_el1_state).

```

```

Definition bmCPU.core c bmRegs := AuthSet.frag (gCPU c) bmRegs.

```

```

Definition lock_step_simulation c ec vRegs :=
  bmCPU.core c bmRegs *
  ec.regs guest ec 1 vRegs *
  [ CPU.related_full_states bmRegs vRegs ].

```

(a) The lock-step simulation of CPU's registers.

```

Axiom IP.bm_reg_step : ∀ c vIpRegs vIpRegs' bmIpRegs,
  lts_step vCPU.IP.lts vIpRegs None vIpRegs' →
  IP.related_full_states bmIpRegs vIpRegs →
  ∃ bmIpRegs' : lts_state (bmCPU.IP.lts c),
  lts_step (bmCPU.IP.lts c) bmIpRegs None bmIpRegs' ∧
  IP.related_full_states bmIpRegs' vIpRegs'.

Axiom IP.bm_msr_step : ∀ vIpRegs vIpRegs' bmIpRegs msrEv,
  lts_step vCPU.IP.lts vIpRegs (Some (vCPU.IP.Msr msrEv)) vIpRegs' →
  IP.related_full_states bmIpRegs vIpRegs →
  ¬MSR.IO.trapped msr →
  ∃ bmIpRegs',
  lts_step (bmCPU.IP.lts c) bmIpRegs (Some (bmCPU.IP.Msr msrEv)) bmIpRegs' ∧
  IP.related_full_states bmIpRegs' vIpRegs'.

```

(b) The axiom we assumed from CPU architects for simulating τ and system register steps of the IP.

Fig. 13. The lock-step simulation can be maintained assuming that “hardware virtualization works”: a hardware virtualized silent (τ) step in the physical IP should have the same semantics with a non-virtualized silent step in the bare-metal IP.

We consider `IP.bm_reg_step` a proof obligation for *CPU architects*. To capture this, our formulation of `IP.bm_reg_step` is part of our architectural model and has no dependencies on our VMM implementation. A proof of `IP.bm_reg_step` should be reusable for any software relying on hardware virtualization.

It is then relatively straightforward to demonstrate that `wp_nova_ec` preserves the lock-step CPU simulation for silent steps of the IP. Effectively, the proof starts by owning `ec.regs guest ec 1 vRegs`, so it can give up this ownership of the register file `vRegs` and instantiate the existential quantifier on the left-hand side of `wp_vcpu_intro` (Figure 12). Then the proof gets back the updated register file `ec.regs is_guest ec 1 vRegs'` and the implementation step `lts_step vCPU.lts vRegs None vRegs'`. If this vCPU step corresponds to an IP step, that is, we have `lts_step vCPU.IP.lts vRegs.(hw_ip_state) None vIpRegs'` where `vRegs' = vReg.(hw_ip_state := vIpRegs')`, then we can use `IP.bm_reg_step` to know that the specification IP can take a similar step. We then lift the specifications IP step to the specification CPU step, and updating the specification state `bmCPU.core c bmRegs` to `bmCPU.core c bmRegs'` where `bmRegs' = bmRegs.(ip_state := bmIpRegs')`. Recall that the proof also knows the states before the step

are related by `CPU.related_full_steps`, so we can easily re-establish the lock-step simulation. This concludes the proof that *one silent step* of the IP preserves the CPU simulation.⁴

Preserving the simulation for τ steps for other CPU's sub-components. For τ steps that come from other components of the CPU (the local interrupt controller or the timer), we need axioms similar to `IP.bm_reg_step` to re-establish the CPU simulation. For internal communications of the CPU that result in τ steps, we also assume similar axioms for τ steps that have no interaction with virtualization, *e.g.*, communication between the timer and the local interrupt controller. Note that in the implementation CPU model, as the MSR component is partitioned out of the vCPU, communication steps between the vCPU's IP and the MSR now make visible events that will be handled separately, in the same way as how we handle the vCPU's external interactions with the rest of the machine, which we discuss next.

5.2.2 The Relaxed Simulation of CPU's Registers, with Visible Events. vCPUs can also perform non- τ -steps, as shown by `wp_vcpu_intro`. Here, the event label `evt` describes the effect on the environment outside the unprivileged CPU execution. Labels `evt.Msr` or `evt.Mem` describe attempts to access privileged resources such as system registers or memory. Depending on hypervisor configuration, such these attempts can succeed, or be blocked. In the success case we rely on axioms such as `IP.bm_msr_step` (Figure 13b) to connect steps in the vCPU semantics to corresponding steps in the bare metal semantics. As with `IP.bm_reg_step`, we view this axiom as an obligation of hardware architects more than an obligation of the VMM since it, effectively, captures the idea that virtualization behaves the same way as non-virtualization when there is no fault.

When the attempt is forbidden, the IP event will record this information making `MSR.IO.trapped` true. This faulting information will be stored within the instruction pipeline until it is either discarded, *e.g.*, because the instruction was an unsound speculation, or it bubbles up to the root of the instruction pipeline. If the instruction reaches the root of the instruction tree, it result in a *trap* which NOVA will handle using `handle_trap` (see Figure 12). We will discuss more about trap handling in Section 5.3.

Faults and Hardware Speculation. Giarrusso et al. [7] discuss how these faults interact with the speculation state of the implementation's hardware, and how this is exposed in the NOVA specification. Figure 14 (borrowed from [7]) illustrates a situation where the speculative execution of the `i2` instruction, which is not at the root of the instruction tree, faults in the vCPU's IP (the second step). However, in our current model, NOVA does not process the fault until the node reaches the root of the instruction tree.⁵ Ultimately, if the IP commits to execution of `i2` (instead of discarding it as unsound speculation), the faulting instruction will reach the root of the instruction tree. At this point, the IP will clear any other instruction instances, and step emitting a `Trap fault_regs` event where the `fault_regs` value is connected to the original fault.

When `i2`'s memory access fails, the vCPU's IP state is exactly one faulting step ahead of the specification's IP state. To support this limited divergence, we must relax the simulation relation between the bare-metal IP and the implementation IP. Specifically, the relation `IP.related_full_states` requires that a bare-metal IP is related to a vCPU's IP if the pipeline structure matches and the instruction instances are pairwise related. The relaxation is now in the relation of instruction instances: two instruction instances are related if they are the same, or the vCPU's IP state is a faulting state where the fault information describes the state of the bare-metal IP. This relaxed simulation can be maintained transparently throughout hardware virtualization steps, until we reach a faulting step.

⁴We must then “recurse” to prove `wp_nova_ec ec` again with the updated state. Informally, we can reapply the same reasoning indefinitely; more formally, we use a form of *coinductive* proof, encoded in Iris via step-indexing and *Löb induction*, but we omit here a formal presentation.

⁵In actual physical hardware execution, the processor is able to speculate into the fault handler, but, for the time being, we assume that this cannot happen. We could enforce this requirement by preventing NOVA from accessing user visible state until after it can guarantee the fault is committed to.

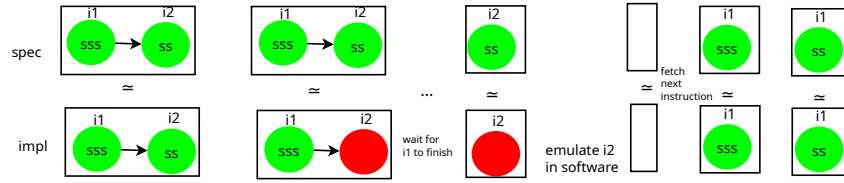


Fig. 14. The simulation between the bare metal (spec) and the vCPU (impl) requires the tree of instructions to match except that the vCPU can be one *faulting* step ahead of the bare metal (figure borrowed from [7]).

Definition $\text{CPU.pendingInstruction } \text{bmRegs } (\text{instr} : \text{DecodeAssist.t}) : \text{Prop} :=$
 $\text{IP.pendingInstruction } \text{bmRegs}(\text{ip_state}) \text{ instr}.$

Axiom $\text{IP.related_full_states_matching_decodes} : \forall \text{ vIpRegs } \text{bmIpRegs } \text{instr},$
 $\text{IP.related_full_states } \text{bmIpRegs } \text{vIpRegs} \rightarrow$
 $\text{vCPU.IP.pendingInstruction } \text{vIpRegs } \text{instr} \rightarrow$
 $\text{IP.pendingInstruction } \text{bmIpRegs } \text{instr}.$

Axiom $\text{IP.mem_trap_step_related} : \forall \{\text{vIpRegs } \text{vIpRegs}'\} \text{bmIpRegs } \text{mem},$
 $\text{Its_step } \text{vCPU.IP.Its } \text{vIpRegs } (\text{Some } (\text{vCPU.IP.Mem } \text{mem})) \text{ vIpRegs}' \rightarrow$
 $\text{IP.related_full_states } \text{bmIpRegs } \text{vIpRegs} \rightarrow$
 $\text{IP.mem_event.trapped } \text{mem} \rightarrow$
 $\text{IP.related_full_states } \text{bmIpRegs } \text{vIpRegs}'.$

Axiom $\text{IP.msr_trap_step_related} : \forall \{\text{vIpRegs } \text{vIpRegs}'\} \text{bmIpRegs } \text{msr},$
 $\text{Its_step } \text{vCPU.IP.Its } \text{vIpRegs } (\text{Some } (\text{vCPU.IP.Msr } \text{msr})) \text{ vIpRegs}' \rightarrow$
 $\text{IP.related_full_states } \text{bmIpRegs } \text{vIpRegs} \rightarrow$
 $\text{MSR.IO.trapped } \text{msr} \rightarrow$
 $\text{IP.related_full_states } \text{bmIpRegs } \text{vIpRegs}'.$

Fig. 15. Axioms on CPU hardware virtualization needed to maintain the IP simulation in case of traps.

Preserving the Simulation. The key to preserving the relaxed simulation lies in transferring the *decode assist information* from the vCPU's IP to the bare-metal CPU's IP. Two principles are necessary here.

- (1) First, we need to ensure that a faulting step does not alter the visible state of the registers in the vCPU. This is true and necessary because the VMM will look at the vCPU registers after the trap to infer information about the bare-metal registers. This preservation property is expressed using the two axioms provided in Figure 15, one for memory actions and the other for system register actions.
- (2) Second, we need to know that faulting instruction instances can only progress via the `Trap fault_regs` event. This is necessary to ensure that the vCPU does not get multiple steps ahead of the bare-metal CPU. `fault_regs` stores the fault registers, which we describe next.

Interpreting Fault Registers/Decode Assists. In the hardware, the reason for a fault to the hypervisor is described by *fault registers* (such as the exception syndrome register (EL2_ESR) [3]). Therefore, we store fault registers as part of the label `Trap fault_regs` of fault steps. Together with the fault type, they encode information such as the faulting address or the privileged state being accessed. For many (but not all) faults, the fault registers provide enough information to fully emulate the faulting instruction, without needing to fetch it from memory and decode it.

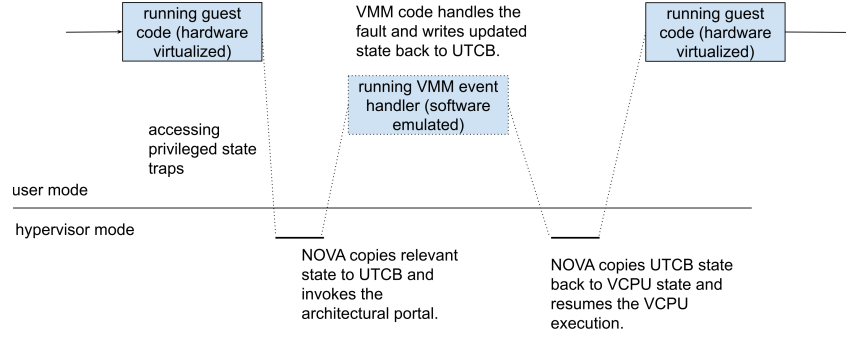


Fig. 16. NOVA's architecture event portal mechanism to forward faults to the VMM.

We connect fault information `faultRegs` to IPs in three steps. First, we connect the fault registers `fault_regs` to a more structured type (`decode_assist.t`). Second, we connect the decode assist to the IP state using `vCPU.IP.pendingInstruction` (for the vCPU's IP state) and `IP.pendingInstruction` (for the bare-metal IP state), and lift the latter to `CPU.pendingInstruction` (for the bare-metal CPU state). These predicates hold only for faulting states, where a *single* instruction is pending, thus `decode_assist.t` only describes a single pending instruction. When a NOVA vCPU emits a `Trap fault_regs` event, NOVA provides us the information in `instr: decode_assist.t` that describes the updated vCPU state (`vCPU.IP.pendingInstruction vIpRegs' instr`), and a proof that `fault_regs` decodes to `instr`. Using the IP simulation and the axiom `IP.related_full_states_matching_decodes` (Figure 15), we can obtain `IP.pendingInstruction bmIpRegs instr`.

Third, we give `decode_assist.t` a meaning: we interpret it to a set of possible behaviors that can be used to *complete* the instruction instance. This forward-looking characterization quotients the intermediate states that might trigger a fault and is directly usable by our proofs. As a concrete example, the meaning of the decode assist `MemRead gpa gpr` includes the trace that reads the memory address `gpa`, stores the result in `gpr`, and increments the instruction pointer by 4. Another example, discussed later, is axiom `IP.pendingInstruction_mrs` in Figure 21. We note that because our model supports spontaneous despeculation, the behavior that does nothing is included in all decode assists.

Transferring the Decode Assist to the Bare Metal. In order to use the information present in the decode assist, we need one final assumption about vCPU transitions labeled by `Trap fault`. Specifically, we need to know that this can only occur when the root of the instruction tree, contains a faulting instruction state labeled *exactly* with `fault`. This assumption, combined with the relaxed simulation relation allows us to prove that when the vCPU transitions via `Trap fault`, the instruction instance at the root of the bare-metal IP can be completed by any behavior permitted by the interpretation of `fault`.

5.3 Handling Virtualization Faults (VMEXITs)

Access to privileged state from the virtual CPU is handled within NOVA. While some of these accesses are virtualized using dedicated virtualization hardware, e.g., 2nd-stage page tables or virtualized MSRs, other accesses cause faults that NOVA delegates to user-space applications through its *architectural portal* mechanism [15, §4.4.2]. Figure 16 shows how NOVA forwards faults from a running execution context to the VMM.

5.3.1 C++ Implementation of Handlers. In the VMM, the handlers for these architectural portals are factored into a common, higher-order function, `call_portal_handler` (see Figure 17 for the skeleton), which handles much of

```

Nova::Mtd
Portal::call_portal_handler(Portal::vcpu_portal_handler portal_handler, const EcCtx* ctx,
                           Vcpu& vcpu, const Nova::Mtd mtd_in) {
    Nova::Utcbs* regs = ctx->utcb();
    Nova::Mtd mtd_out;

    vcpu.begin_emulation();
    vcpu.update_inj_status(regs); // reconcile interrupt state
    portal_handler(vcpu, regs, mtd_in, mtd_out); // fault-specific logic
    vcpu.reconfigure(regs);
    vcpu.end_emulation();
    vcpu.inject_irqs(regs); // update interrupt state

    return mtd_out;
}

```

Fig. 17. The high-level structure of the generic architectural portal handler includes: register parsing, reading and updating the state of the local interrupt controller, and portal reconfiguration.

the common infrastructure around CPU coordination, CPU reset, and interrupts, and delegates fault-specific logic to the function pointer argument `portal_handler`.

The `call_portal_handler` has access to a subset of architectural state, represented by UTCB `regs` and a *Message Transfer Descriptor* (MTD) `mtd` delineating the concrete subset for the given portal, as well as access to C++ representation of the vCPU `vcpu`.

- First, `call_portal_handler` enters the emulation section of execution by calling `vcpu.begin_emulation()`. This is a point of synchronization in case the VMM is seeking to pause execution of the guest.
- Next, the handler reads the state of the local interrupt controller—the GICC in ARM—and updates the global interrupt controller to reflect which interrupts have been processed.
- Then, the concrete handler is called. Note that the handler is treated opaquely and simply provided with C++ representations of virtual CPU (`vcpu`), architectural state (in form of UTCB `regs` and the input MTD `mtd_in`) and is expected to update the register representation and provide appropriate `mtd_out`.
- Since a handler can request changes to vCPU features (e.g., enable or disable single stepping), we need to reconfigure virtualization control registers in `vcpu.reconfigure(regs)`.
- Subsequently, the handler exits the emulation section, and updates the interrupt state with the call to `vcpu.inject_irqs(regs)`.
- It is useful to note that the state of the vCPU does not change until the function returns, at which point NOVA will update the vCPU state using the returned MTD and UTCB. This delayed reconciliation means that the simple lock-step relation between the specification CPU and the NOVA vCPU (Section 5.2.1) is broken throughout this function.

5.3.2 The Specification of `call_portal_handler`. Figure 18 presents the key components of the specification for the `call_portal_handler` function for general-purpose registers. (We return to system registers in Section 5.5.)

- Lines 2 to 5 identify arguments of the function and lines 6 to 12 carry the C++ state (including ownership of the portal-specific callback at `fp_ptr`) and identify that the spec has access to NOVA context, which is relatively straightforward.
- Lines 14 to 16 convey the lock-step simulation that held during the normal execution before the fault, which we pick up and use to re-establish the simulation. We note that the ownership of `ec.regs` `ec_vRegs`

```

1 cpp.spec (named "call_portal_handler") as call_portal_handler_spec with (
2   \arg{fptr} "portal_handler" (Vptr fptr)
3   \arg{ctx} "ctx" (Vptr ctxp)
4   \arg{vcpu} "vcpu" (Vptr vcpu)
5   \arg{amtd} "mtd_in" (Nova.MTDs.to_V mtd_in)
6   \persist fptr ↦ cpp_specR callee_type call_portal_handler_callee_spec
7
8   \nova pd ec
9   \pre{vRegs tsrRegs faultRegs ctrlRegs}
10    ctxp ↦ EcCtxR 1 (utcbR mtd_in vRegs tsrRegs faultRegs ctrlRegs)
11   \pre{γvcpu m} vcpu ↦ Vcpu.R 1 γvcpu m
12   \prepost{q m} _global "Model::vm_asl" ↦ ptr_toR (Tnamed "Model::AsLayout") q (AS_LayoutR q m)
13
14   \prepost ec.reg ec vRegs
15   \pre{c bmRegs} bmCPU.core c bmRegs
16   \require CPU.related_full_states bmRegs vRegs
17
18   \require{instr} vCPU.FaultRegs.encoded_decodeassist faultRegs instr
19   \require CPU.pendingInstruction bmRegs da
20
21   \prepost ec.ctrl_regs ec ctrlRegs
22   \prepost ec.trappable_msrs ec tsrRegs
23   \persist vCPU.CtrlRegs.I c ctrlRegs
24   \require msr_simulation ctrlRegs bmRegs.(msrs) tsrRegs m.(msrVals))
25
26   \post{mtd_out} [Nova.MTDs.to_V mtd_out]
27    ∃ bmRegs' vRegs' tsrRegs' ctrlRegs' msrVals',
28    ctxp ↦ EcCtxR 1 (utcbR mtd_out vRegs' tsrRegs' faultRegs ctrlRegs')
29    * [ cpu_is_isa_stable vRegs' ]
30    * ec.trappable_msrs ec tsrRegs'
31    * vCPU.CtrlRegs.I c ctrlRegs'
32    * vcpu ↦ Vcpu.R (cQp.m 1) γvcpu m.(msrVals' . = msrVals')
33    * bmCPU.core c bmRegs'
34    * [ CPU.related_isa_states bmRegs' vRegs' ] *
35    * [ CPU.pendingInstruction bmRegs' DecodeAssist.EmptyPipeline ] *
36    * [ msr_simulation ctrlRegs' bmRegs'.(msrs) tsrRegs' msrVals' ]
37 ).

```

Fig. 18. The specification of the generic call_portal_handler.

is preserved across the function (using the `\prepost` modifier) which reflects the fact that this state is only updated by NOVA until after the handler returns to NOVA.

- The subtlety of the specification lies in the next two lines, 18 and 19, which characterize the reason for the fault in the bare-metal state. As discussed in [Section 5.2.2](#), the decode assists information `instr` characterizes possible behaviors to complete the faulting instruction. While the virtualization hardware has rolled back the execution of the faulting instruction and presented it to the fault handler through the decode assist information, we have deliberately not rolled back the bare-metal CPU state yet. This allows us to simulate the instruction without needing to completely restart it, which would be time consuming and complex since the faulting instruction may not even still be in memory at the program counter. Nonetheless, the `portal_handler` callback can rollback the instruction if needed, and either emulate it in its entirety or ask the virtualization hardware to re-try it.

In either case, the handler needs to ensure that the hardware virtualization lock-step simulation ([Section 5.2.1](#)) will hold after NOVA updates the vCPU state according to the returned MTD and UTCB.

- In the post-condition, the specification returns updated C++ ownership. Note that the updated state of Vcpu class `m.(msrVals := msrVals')` carries new values `msrVals'` of (trappable) system registers, which might have been updated by the handler. The updated bare-metal state is returned through `bmCPU.core c bmRegs'` and lines 34 to 36 of the post condition reflect, somewhat verbosely, that the state of the vCPU after NOVA interprets the MTD and UTCB will coincide with the new bare-metal state (`bmRegs'`).

It is worth noting the the vCPU state must be an ISA-stable state (`[ cpu_is_isa_stable vRegs' ]`) which characterizes that the full state of the CPU can be determined from only the state visible in the ISA specification. Specifically, this implies that the vCPU's IP is empty. This seems reasonable since, in hardware, its state is serialized to memory while a virtual CPU is not running.

5.4 The System Register Simulation

The handling of system registers (MSRs on x86) is more nuanced than the handling of general purpose registers. Because some interactions with system registers have impacts outside the CPU, hardware provides mechanisms to intercept *direct* accesses to these registers,⁶ and some of these registers have no hardware emulation support, *i.e.*, they are purely software emulated. Based on the virtualization features provided by the hardware, we partition the CPU registers into the following categories.

- **Pure Hardware Virtualized** registers are completely virtualized by hardware and accesses to them cannot be trapped on. Most recognizable amongst these registers are general purpose registers, but the timer register is also included in this set. NOVA provides access to these registers through the UTCB and MTD. As we saw previously, the simulation relation for these registers will connect the vCPU state directly to the bare-metal state, and C++ will get views of this data when handling VMEXITS.
- **Pure Software Emulated** registers have no hardware virtualization support and accesses to them always results in a VMEXIT. Because there is no hardware virtualization support, the implementation state of these registers lives entirely within C++.
- **Trappable** registers are a hybrid of hardware virtualized and software emulated registers. For these registers, NOVA exposes control registers that dictate what types of accesses to these registers trigger faults. For example, setting the `HCR_EL2.TVM` bit of hypervisor configuration register (which NOVA exposes in the UTCB) will cause changes to the `TTBR_EL1` register to fault. The handling of these registers is often *ad hoc*, highly dependent on the register, and sometimes changes over the lifetime of the CPU depending on the value of the control register.

5.4.1 System Register Semantics. While called “registers”, system registers often have semantics that deviate from the traditional get-set semantics of general purpose registers. Some examples include,

- the pair of `CSSELR` and `CCSIDR` registers for selecting cache size - writing a value to `CSSELR` changes synchronously both the registers
- `CurrentEL` that simply extracts the relevant part of `PSTATE`, and

To encapsulate the behavior of system registers, the set of which can grow with new architecture releases, we model them as a set of individual state transition systems separate from the general-purpose registers. These transition systems are parallel composed together as the MSR component in [Figure 11](#). To give an example, [Figure 19](#) demonstrates the semantics of a simple get-set register that may or may not be writable. To date, we

⁶Hardware semantics also includes *indirect* accesses which cover access that are implicit dependencies of an instruction which are generally not interceptible. One good example of an indirect register access is a read of the page table root register during an address translation.

```

Inductive step {reg_id: idT} {writable: bool} {write_f: N → N → N}
  : State → option IO.t → State → Prop :=
| MSR_READ direct s :
  step s (Some { I0.ix := reg_id; I0.faultRegs := None
                ; I0.evt := Read direct (Some s.(value)) }) s
| MSR_WRITE s v : writable = true →
  step s (Some { I0.ix := reg_id; I0.faultRegs := None
                ; I0.evt := Write true v false }) s.(value %= write_f v).
(* ^^^^ no indirect writes *)

```

Fig. 19. The single step function of the STS for a simple get-set register that may or may not be writable.

```

Definition msr_simulation ctrlRegs bmRegs vRegs cpp : Prop :=
  ∀ msr_id: MSR.idT,
  match register_sim_type ctrlRegs msr_id with
  | SwEmulated ⇒ (* Pure Software Emulated *)
    cpp.(bus_rmap) !! msr_id = Some (bmRegs.(bm_esr).(ESR.reg) msr_id)
  | TrappableHwVirt ⇒ (* Trappable, currently hardware virtualized *)
    vcpuRegs.(vcpu_tsr).(TSR.msr_reg) msr_id = bmRegs.(bm_tsr).(TSR.msr_reg) msr_id
  | TrappableTrapped ⇒ (* Trappable, currently software emulated *)
    vcpuRegs.(vcpu_tsr).(TSR.msr_reg) msr_id = bmRegs.(bm_tsr).(TSR.msr_reg) msr_id ∧
    cpp.(bus_rmap) !! msr_id = Some (bm.(bm_tsr).(TSR.msr_reg) msr_id)
  | ... (* custom simulations for more complex registers omitted *)
  end.

```

Fig. 20. The simulation for pure software emulated and trappable system registers.

have carried out proofs for several representative registers, but, as mentioned previously, more work is needed to cover the full range of system registers.

5.4.2 The Simulation of System Registers. The simulation relation for pure software emulated and trappable system registers is captured by the relation `msr_simulation`, whose excerpt is shown in Figure 20. Note that the `register_sim_type` function classifies each register (`msr_id`) using the values of the control registers (`ctrlRegs`), allowing the simulation for trappable registers to change over time. The elided cases in this definition capture more subtle simulations for complex registers, such as the timer register.

One particularly subtle simulation relation is of the single stepping register (MDSCR): single stepping must be enabled in both the guest and the hypervisor level in order to activate single stepping. Since MDSCR_EL1 is a user-visible register, if the VMM wants to single-step the guest, it must take care that the guest does not observe the value in this register, which would deviate from the bare-metal value. We are still working on crafting an appropriate simulation here, but, in the meantime, note that single stepping a guest is not a feature that is crucial for a basic VMM implementation, though advanced security features of the BlueRock VMM do rely on this feature.

5.5 Handling System Register Faults

As discussed in Section 5.4, the generic `call_portal_handler` function delegates the event-specific portion of the trap to an event-specific callback. The `portal_handler` callback re-establishes the CPU simulation in order to resume hardware virtualization. In case of MRS (read) and MSR (write) events, that means completing the instruction for the trapped register access. The `IP.pendingInstruction_mrs` axiom, shown in Figure 21, is needed

```

Axiom IP.pendingInstruction_mrs :  $\forall$  srcMsr dstGpr bmIpRegs,
  IP.pendingInstruction bmIpRegs (DecodeAssist.Mrs dstGpr srcMsr)  $\rightarrow$ 
 $\forall$  v,  $\exists$  bmIpRegs' tr,
  lts_step_star bmCPU.IP.lts bmIpRegs tr bmIpRegs'  $\wedge$ 
  erase_tau tr = [vCPU.Msr (MSR.IO.Mk srcMsr None (MSR.Read true (Some v)))]  $\wedge$ 
  IP.pendingInstruction bmIpRegs' DecodeAssist.EmptyPipeline  $\wedge$ 
  IP.same_regs bmIpRegs' bmIpRegs.(reg IP.PC %= N.add 4).( gprExt dstGpr .= v).

```

Fig. 21. The axiom needed to update the bare-metal IP state after faulting, MRS case.

```

1 static void
2 system_register(Vcpu::Vcpu& vcpu, Nova::Utcba arch, Nova::Mtd mtd_in, Nova::Mtd& mtd_out) {
3   const Esr::MsrMrs msr_info{arch  $\rightarrow$  el2_esr()};
4   Msr::Err e = msr_info.write() ? vcpu._msr_bus.write(msr_info, arch, mtd_out)
5                               : vcpu._msr_bus.read(msr_info, arch, mtd_out);
6   switch (e) {
7   case (Msr::Err::OK):
8     arch  $\rightarrow$  advance_pc();
9     break;
10  default:
11    INFO("MSR/MRS access error %s", msr_info.info());
12    arch  $\rightarrow$  inject_undefined_exception();
13  }
14 }

```

Fig. 22. A (simplified) system register event handler in C++.

to interpret the MRS decode assist and it is a concrete instance of the “semantics of decode assists” that we discussed in [Section 5.2.2](#). This reasoning principle is the key step in the proof of MSR/MRS handler code. It allows us, after properly stepping the bare-metal CPU’s register (using the step relation in [Figure 19](#)), to conclude that the instruction pipeline is empty, which is required by the post-condition of `call_portal_handler_spec` ([Figure 18](#)).

5.5.1 System Register Handler. The actual code in the BlueRock VMM that handles MRS and MSR events is generic and handles several other hardware events as well (for example reading and writing 32-bit special-purpose registers, which come in several flavours). For the sake of simplicity we present a highly stylized version of MSR handling code ([Figure 22](#) and [Figure 24](#)) which, nevertheless, demonstrates the important steps in C++ as well as the core proof steps that occur in the verification of the real implementation.

The top level callback is the `system_register` function ([Figure 22](#)). This function is registered as a portal entry point callback for architectural events that correspond to reads and writes of system registers. The callback first parses the decode assist from the exception syndrome register (ESR_EL2) ([Figure 22](#), line 3). This information captures whether the access is write or read, and the IDs of the source system register and the target general purpose register (the code reuses these mnemonics for both reads and writes). Based on the event the function delegates the handling to the `vcpu` object (in [Figure 22](#), we inline the handling for brevity, lines 4 and 5). If the call succeeds, the specification of the accessing functions guarantees that all that remains to re-establish the simulation for hardware virtualization is to advance the program counter which occurs on line 8. If it fails, the code injects the undefined system register exception into the architectural UTCB ([Figure 22](#), line 12).

5.5.2 MSR Bus and Register Emulators. Due to the large number of system registers that vary between different platforms, system register emulation is organized in a bus that maps register identifiers to corresponding register emulators. This design makes it easy to selectively enable or disable registers and to add new registers to support more hardware features. This indirection within the implementation makes it possible to verify the `system_register` function without needing to know the specific register that is invoked, *i.e.*, the proof does not depend on case analysis on the particular register.

Achieving this level of indirection requires a uniform interface to all of the system register emulators, including unifying both purely software and trappable system registers which rely on very different simulations (see Figure 20). Figure 23 shows a slightly simplified version of the specification for the write method of the `Register` class. In this specification, lines 10 to 13 capture C++ ownership of the system register state, and lines 15 to 17 convey the lock-step simulation before the fault. The success-case post-condition on lines 21 to 32 expresses the updated ownership, *almost* re-establishing the simulation relation. The “almost” here is due to the delayed update of the program counter (Figure 22, line 8). This is captured by the `let`-declaration on line 30 (Figure 23) which binds `vRegs'` *after* it has been updated; however, the value of `vRegs` in the UTCB (line 25) does not have the updated program counter yet. Concretely, the predicate `mtd_covers_diff vRegs' (mtd_out' ∪ [Nova.Mtd.EL2_EL2_SPSR]) bmRegs bmRegs'` will allow the caller, *i.e.*, ultimately the `call_portal_handler`, to reason that as long as UTCB contents are updated to `vRegs'` and the returned `mtd_out'` is extended with the flag corresponding to the EL2 register, the architectural state that will be restored by NOVA is in simulation with bare-metal state.

The error case on lines 33 to 40 returns C++ ownership and does not update the bare-metal state. The concrete error cases (`PError`) are abstracted over as they depend on concrete types of registers and are only provided later, when the concrete registers are specified. In this failure case, stepping the bare metal is delayed until the injection of exception in `system_register` (Figure 22, line 12).

5.5.3 Verifying the Register Emulators. Register emulators are implemented as sub-classes of the `RegisterBase` class. For example, the default implementation of the write method in the `Register` class (Figure 24) implements the behavior of a purely software emulated register that follows the specification from Figure 19. Recall that in the purely software emulated case of the simulation, the simulation relation connects C++ state directly to the bare metal state, so only C++ state in the MSR bus needs to be updated. The implementation performs this update on line 8. This update is justified by combining the `IP.pendingInstruction_msr` axiom (the write-analog of `IP.pendingInstruction_mrs` shown in Figure 21) with the operational semantics of this register (again, Figure 19) which discharges stepping of the MSR state. As in `IP.pendingInstruction_mrs`, `IP.pendingInstruction_msr` updates the program counter state which brings us to the “almost” simulation state discussed in Section 5.5.2.

The more interesting implementation is in the `Wtrapped` subclass of `Register` which handles trappable system registers. Recall that, when the register is trapped, the MSR simulation covers the bare-metal state, the state in the vCPU (which during the handler is stored in the UTCB), *and* the state in the system register emulator. Updating the register emulator state is performed on line 19 through the call to the default implementation, and then, lines 21 to 32 must update the UTCB and MTD to preserve the simulation when the vCPU is resumed. It is subtle, but important to note that this simulation means that if, at any point, software trapping of a trappable register is disabled, then when it is re-enabled, the current value of the register from the vCPU (again via the UTCB) must be copied into the MSR emulator in order to re-establish the MSR simulation.

Future work. We have verified the code on path from `call_portal_handler` through `system_register` to the call to individual register emulator access; however, we have verified only several cases of individual register emulators. We have verified emulators covering the division in Section 5.4 and we thus believe we have validated the described specification and feasibility of the approach. In future, it would be beneficial to verify various special cases of register emulators, especially those where several registers share state (e.g. `CSSELR` and `CCSIDR`).

```

1 Definition write_interface PError (RegR : cQp.t → MSR.idT → MSR.RegMap → Rep) :=
2   (virtual.vptrI
3     { %cpp "Msr::RegisterBase::write(Esr::MsrMrs&, Nova::UtcB*, Nova::Mtd&)" }
4     (TT := [tele (_ : cQp.t) (_ : MSR.idT) (_ : MSR.RegMap)]) RegR $
5
6     \arg{msr_infor} "msr_info" (Vref msr_infor)
7     \arg{archp} "arch" (Vptr archp)
8     \arg{mtd_out} "mtd_out" (Vref mtd_out)
9
10    \pre{reg_map} this ↦ R 1 reg_id reg_map
11    \pre{vRegs tsrRegs faultRegs ctrlRegs} archp archp ↦ utcbR mtd_in vRegs tsrRegs faultRegs ctrlRegs
12    \prepost{gpr_reg} msr_infor ↦ MsrMrsR reg_id gpr_reg
13    \pre{mtd_out} mtd_out ↦ MtdR mtd
14
15    \pre{c} bmCPU.core c bmRegs
16    \require{da} CPU.pendingInstruction bmRegs (DecodeAssist.MsrMrs da)
17    \require{all_tsrs} msr_simulation bmRegs all_tsrs reg_map
18
19    \post{err}[Err.to_V err]
20    match err with
21    | Err.OK ⇒ ∃ bmRegs' all_tsrs' v_masked,
22      let reg_map' := <[reg_id := v_masked]> reg_map in
23      this ↦ R 1 reg_id reg_map'
24      * let tsrRegs' := update_if_trappable tsrRegs reg_id v_masked in
25        archp ↦ utcbR mtd_in vRegs tsrRegs' faultRegs ctrlRegs
26      * let mtd_out' := mtd_out ∪ {[Nova.Mtd.from_reg_id reg_id]} in
27        mtd_out ↦ MtdR mtd_out'
28      * bmCPU.core c bmRegs'
29      * [ CPU.pendingInstruction bmRegs' DecodeAssist.EmptyPipeline ]
30      * let vRegs' := vRegs %≡ advance_pc in
31        [ mtd_covers_diff vRegs' (mtd_out' ∪ {[Nova.Mtd.EL2_EL2_SPSR]}) bmRegs bmRegs' ]
32      * [ msr_simulation bmRegs' all_tsrs' reg_map' ]
33    | Err.ACCESS_ERR ⇒
34      this ↦ R 1 reg_id reg_map
35      * archp ↦ utcbR mtd_in vRegs tsrRegs faultRegs ctrlRegs
36      * mtd_out ↦ MtdR mtd_out
37      (* in error case, we do bare metal step in inject exception *)
38      * bmCPU.core c bmRegs
39      * PError reg_id
40      (* ^^^ violated a derived-specific condition (e.g. register is read only) *)
41    | _ ⇒ ...
42  end).

```

Fig. 23. The virtual method specification for RegisterBase::write.

6 THE MEMORY SUBSYSTEM

In this section, we explain how we are proving that the VMM's memory component refines the bare-metal memory. First, [Section 6.1](#) reviews how communicating steps, such as those between the CPU and the memory, update the bare-metal state in our separation logic. From [Section 6.2](#) we define, and incrementally extend, the simulation for memory, that is, the *memory invariant*, while discussing how the VMM maintains it.

```

1 Err Register::write(Esr::MsrMrs const& msr_info, Nova::Utcb
  * arch, Nova::Mtd& mtd_out) override {
2   if (! _writable) return Err::ACCESS_ERR;
3   // Writing 1 to a res value, error out
4   if (value & ~_write_mask) return Err::ACCESS_ERR;
5
6   value = arch→gpr(msr_info.target_reg());
7   _value |= value & _write_mask;           // Set
      the bits at 1
8   _value &= ~(~_write_mask & (value ^ _write_mask)); // Set
      the bits at 0
9   return Err::OK;
10 }
11
12 Err Register::read(Esr::MsrMrs const& msr_info, Nova::Utcb
  arch, Nova::Mtd& mtd_out) override {
13   uint64 value = _value;
14   arch→gpr(msr_info.target_reg(), value);
15   return Err::OK;
16 }
17
18 Err Wtrapped::write(Esr::MsrMrs const& msr_info, Nova::Utcb
  * arch, Nova::Mtd& mtd_out) override {
19   Err err = Register::write(msr_info, arch, mtd_out);
20   if (err != Err::OK) return err;
21   switch (msr_info.id()) {
22     case (Msr::SCTLR_EL1):
23       arch→el1_sctlr(reg_value);
24       mtd_out |= Nova::MTD::EL1_SCTLR;
25       break;
26     case (Msr::TTBR0_EL1):
27       arch→el1_ttbr0(reg_value);
28       mtd_out |= Nova::MTD::EL1_TTBR;
29       break;
30     case ... :
31       ... // remaining write-trapped registers in UTCB
32   }
33
34   return Err::OK;
35 }

```

Fig. 24. Register Emulator implementations.

6.1 Updating the Bare-Metal State with Communicating Steps

In our bare-metal whole-machine specification, physical memory is a passive component that is only accessed by either CPUs or by devices (through DMA). In the formal model (Section 3) using labeled transition systems (LTSes), this means that all memory steps with visible events cancel with steps of matching events from CPUs or devices. For instance, a CPU can read or write a value from some (guest) physical address gpa . We model that via a step for the memory LTS and a “dual” step for the CPU LTS. If the memory access is initiated by some device, the dual event will be for that device, but in this section we will focus on accesses initiated by CPUs.

In the universal Event type presented in Figure 6, a read event on the memory will have the label $ev_c := \text{Read Completer Memory } gpa \text{ bytes_sz } 0 \text{ (ReadSync (Some } v))$, for reading $bytes_sz$ bytes from guest physical address gpa and obtaining value v . Here, the address and access size are picked by the Requester n_r (the CPU or device), while the value is picked by the Completer (memory). Similarly, a write event on the memory will have the label $ev_c := \text{Write Completer Memory } gpa \text{ bytes_sz } 0 \text{ } v' \text{ (WriteSync false)}$ for writing the value v' . On the side of the Requester n_r , we will have a request step ev_r that cancels with ev_c .

Our ownership of the bare-metal state provided by Spectra lets us step our specification machine to maintain the simulation with the implementation machine, and enables us to decompose the proof by components. Furthermore, Spectra allows us to update the states of the two bare-metal components that are performing matching communicating steps. For example, we can update the bare-metal memory state with the ownership $\text{AuthSet.frag } \gamma_{\text{mem}} m$ in tandem with (logically atomic with) updating the bare-metal state of the Requester, *e.g.*, using $\text{bmCPU.core } c \text{ bmRegs}$ for the Requester CPU.

These two updates of the two communicating components are encoded as two dependent separation logic obligations. On the one hand, the Requester of the event ev_r needs to show a *committer* $\text{committer } \text{bmMEM.lts } E \gamma \{ \llbracket ev_c \rrbracket \} Q$ for the Completer’s dual event ev_c (see Section 2.3). When ev_r is a CPU event, proving this committer requires updating $\text{bmCPU.core } c \text{ bmRegs}$ via step ev_r and maintaining any associated simulation (*e.g.*,

```

Definition memory_inv_v1 :=
  inv memory_ns (
    ∃ m gpa_to_hpa,
    (* whole bare-metal memory ownership *)
    AuthSet.frag γmem m * bm.bytes_at_auth (to_map m) * ... *

    (* second-stage page table ownership, for hardware virtualization *)
    space_page_mappings GST gpa_to_hpa *

    (* related hardware bytes and fictionally separated bare-metal bytes *)
    [*list] gpa ∈ dom (to_map m), ∃ n, bm.byte_at gpa 1 n * hw.byte_at (gpa_to_hpa gpa) 1 n
  ).

```

Fig. 25. The first version of the memory invariant: 2nd-stage page table mappings, and related bare-metal bytes and physical (hardware) bytes, with the bare-metal memory fictionally separated.

the CPU simulation). This update can produce some resource Q , which will be returned to callers. Each Requester with the event ev_c can perform this proof on its own, without needing to depend on the memory invariant.

On the other hand, eliminating the committer lets us update $AuthSet.frag \gamma_{mem} m$ according to memory event ev_c , and produce resource Q . When performing this update, we must prove that the update also maintains the simulation associated with the Completer, which, for the memory component, is the memory invariant. In combination, we maintain the simulation for the whole step of the whole machine in a communicating step by separately maintaining the corresponding simulations of the two participating components.

6.2 The Basic Memory Simulation

The memory invariant defines the simulation relation linking the specification state to the implementation state for the memory component. Intuitively, the invariant requires that each byte of the bare-metal memory has the same content as the corresponding byte of the implementation memory. The implementation memory can be concurrently accessed by both CPUs and devices through both hardware virtualization and software emulation. Furthermore, the invariant must also support changing the relationship over time, *e.g.*, to support introspection features that “watch” for accesses to certain regions of memory. We start with the first simple version (v1) of the invariant in Figure 25.

- The function `gpa_to_hpa` maps each guest physical address (GPA) `gpa` to a corresponding host physical address (HPA). In the implementation, these mappings are connected to the guest page tables which are managed by NOVA and encapsulated in the ownership `space_page_mappings`. This ownership is needed for hardware virtualization of guest memory accesses, which we will discuss in Section 6.3.
- The assertion `bm.byte_at gpa q n` represents ghost ownership of the *bare-metal* byte at the GPA `gpa` of the bare-metal memory, with the current value `n`.
- The assertion `hw.byte_at hpa q n` represents ownership of the *physical* byte at the address `hpa` of the physical memory, with the current value `n`.

Memory ownership is held in an invariant because memory can be accessed concurrently by multiple threads—by guest CPUs, emulated devices, and passthrough devices. We need not impose any further sharing protocol on memory, beyond refining bare-metal memory: all possible VMM execution traces refine bare-metal traces. Informally, any concurrency “bugs” can be blamed on the unverified guest.⁷

⁷We currently assume the sequential consistency memory model and ignore weak memory, but our physical memory would simulate bare-metal memory in lockstep even if we switched to a more complex model for memory state.

Separation logic for the bare-metal memory. The assertion `bm.byte_at` is not exposed directly from our bare-metal model: what we have to begin with the simulation from the Spectra library is the ghost ownership of the whole bare-metal memory component `AuthSet.frag  $\gamma_{\text{mem}}$  m`. We therefore need to apply fictional separation to this big-footprint ownership to get per-byte ownership of the bare-metal memory. Using a resource algebra for maps [8], we define a ghost variable with assertions `bm.bytes_at_auth (bytes : gmap addr N)` and `bm.byte_at gpa q (n : N)`, such that `bm.bytes_at_auth bytes * bm.byte_at gpa q n  $\vdash$  bytes !! gpa = Some n`, and with the appropriate rules for updating the values of individual bytes. The memory invariant then simply links `bm.bytes_at_auth (to_map m)` and `AuthSet.frag  $\gamma_{\text{mem}}$  m` with the same ⁸.

6.3 Hardware Virtualization of vCPUs for Accessing Memory

We discuss how the memory invariant is maintained by hardware virtualization, through the ownership of the 2nd-stage page tables. We focus on vCPUs and their semantics given by rule `wp_vcpu_intro` (Figure 12).

In Figure 12, NOVA defines the obligations for attempts to access memory (or to perform memory-mapped I/O, see Section 8) with a memory event `evt.Mem mem` in the weakest pre-condition style, through the predicate `handle_mem ec mem`. Consider the case where the event `mem`, sent from the CPU’s instruction pipeline, is `IP.MemRequest (IP.MemRequest.mk gpa bytes_sz r should_trap aperm)`.⁹ Here, the CPU is attempting to perform an access requester `r` (a read or write) to address range `[gpa, gpa + bytes_sz)`. (We discuss `aperm` and `should_trap` in a moment.)

Figure 26 provides some details of `handle_mem ec mem`, which encodes the obligations that the NOVA specification requires us to show in order to make a memory access step on the vCPU of `ec`. To process the memory event, we must first provide the resources needed by NOVA to perform 2nd-stage address translation via `wp_nova_ec_opte_lookup` (elided). The hardware CPU will consult 2nd-stage page tables to translate the *guest* physical address `gpa` to a host physical address and check if permissions are sufficient for the access. We use the page-table mappings `space_page_mappings` from the memory invariant, which give enough resources to safely reading any page-table entry. In other words, `space_page_mappings` should be enough to justify `wp_nova_ec_opte_lookup`, allowing the vCPU implementation to make this translation step.

The results of the translation include the permissions `pt_perms` and an optional *host* physical address `ohpa`, as well as the boolean value `should_trap`. These results must come from the values stored in `space_page_mappings` in the memory invariant. If `gpa` is unmapped, or mapped with insufficient permissions, translation fails (`ohpa = None`) and this vCPU step must be a *failing* access attempt, where `should_trap = true`. Moreover, we also know that `aperm = pt_perms`, that is, the fields `aperm` and `should_trap` are *output* values in the IP’s memory event `mem`, which NOVA makes available to us from the translation results. As discussed in Section 5.2.2, the instruction attempting the memory access is “paused” until it is either despeculated, or until it comes to the top of the instruction tree and causes a trap. This is invisible in the bare-metal CPU, so, in order to maintain the *CPU simulation*, we apply the axiom `IP.mem_trap_step_related` (Figure 15). In this case, the memory invariant is only read and not modified, and the memory simulation is thus still maintained.

If `gpa` is mapped, then translation succeeds (`ohpa = Some hpa, should_trap = false`), and the CPU will proceed with the access, either through the main memory (DRAM) or to a device. The NOVA specification requires us to show `do_mem_or_mmio_read` or `do_mem_or_mmio_write` for a read or a write respectively. To do so, we borrow the resources we have from the memory invariant. For now, we focus on accesses to DRAM, which the NOVA specification ensures are synchronous. Our proof can now borrow the physical bytes (`hw.bytes_at`) targeted by

⁸Admittedly, introducing `bm.byte_at` is not strictly necessary for the invariant as shown, since it stores both `AuthSet.frag` and `hw.byte_at` and could relate them directly. However `bm.byte_at` is more flexible and becomes necessary for the full invariant.

⁹This event will be turned into the universal Event type, e.g., as `Read Requester Memory gpa (bytes_sz * 8) 0 (ReadSync (Some v))` to be sent to be the memory.

```

Definition do_mem_or_mmio_read (hpa : paddr) voffset (bytes_sz : N) rr should_trap (Q : mpred) : mpred :=
  match rr with
  | IP.ReadSync r => ∃ r', (∃ q, hw.bytes_at (hpa + voffset) bytes_sz q r' * True) ∧ ([ r' = r ] -* Q)
  | ... (* handling of asynchronous accesses; only relevant for devices *)
  end.

Definition do_mem_or_mmio_write (hpa : paddr) voffset (bytes_sz : N) val wr should_trap (Q : mpred) : mpred :=
  match wr with
  | IP.WriteSync =>
    (∃ r, hw.bytes_at (hpa + voffset) bytes_sz 1 r) * (hw.bytes_at (hpa + voffset) bytes_sz 1 val -* Q)
  | ... (* handling of asynchronous accesses; only relevant for devices *)
  end.

Definition handle_mem ec mem :=
  match mem with
  | IP.MemRequest (IP.MemRequest.mk gpa bytes_sz r should_trap aperm) =>
    let voffset := page_offset_of gpa in
    letI* pt_perms, ohpa :=
      wp_nova_ec_opte_lookup (↑nova_ns) is_guest hst ec is_guest gpa bytes_sz
      (IP.MemReq.perms_sufficient_for r)
    in
    [ aperm = pt_perms ] -*
    let Q := [↑nova_ns, T] => wp_nova_ec ec in
    match ohpa with
    | None => (* not mapped or insufficient permissions, access fails *)
      [ should_trap = true ] -* Q
    | Some hpa =>
      [ should_trap = false ] -*
      match r with
      | IP.ReadRequest rr acc_type =>
        do_mem_or_mmio_read hpa voffset bytes_sz rr should_trap Q
      | IP.WriteRequest val wr =>
        do_mem_or_mmio_write hpa voffset bytes_sz val wr should_trap Q
      end
    end
  | IP.MemResponse ... => ... (* handling of asynchronous accesses; only relevant for devices *)
  end.

```

Fig. 26. An excerpt of the vCPU’s obligations enforced by NOVA for memory accesses.

the access from the invariant, and return its (possibly) updated contents to the invariant. For simplicity, assume we are writing a single byte. We perform this step using lemma `memory_inv_update_single_byte` in [Figure 27](#). Here, `gpa_translates_to_hpa gpa hpa` is a ghost assertion showing that `gpa` translates to `hpa`, linked with the `space_page_mappings` conjunct of `memory_inv_v1`. Crucially, lemma `memory_inv_update_single_byte` only depends on the memory invariant, but does not make assumptions on the initiator of the memory event: hence, this part of the proof can be reused for events initiated by devices.

To conclude the proof via `memory_inv_update_single_byte`, we must prove that we can step the bare-metal memory component via a committer on memory (as discussed in [Section 6.1](#)). We can prove this via `cpu_dram_committer_intro`, which builds a CPU requester via `cpu_requester_intro` and transports it to a memory committer via `transport_singleton`. Finally, applying `cpu_dram_committer_intro` requires a dual bare-metal CPU step. We prove that by requiring the axiom `IP.bm_sync_mem_step`—“hardware virtualization works for synchronous memory steps”, given in [Figure 28](#).

```

Lemma memory_inv_update_single_byte dn E gpa hpa γ v_new Q :
  let memEv := Write Completer Memory gpa 1 0 v_new (WriteSync false) in
  E ## memory_ns →
  memory_inv_v1 *
  gpa_translates_to_hpa gpa hpa *
  committer bmMEM.lts E γ {[ memEv ]} Q ⊢
  AC <<< ∃∃∃ n, hw.byte_at hpa 1 n >>> T, T \E \memory_ns <<< hw.byte_at hpa 1 v_new COMM Q >>>.

Lemma cpu_requester_intro γ E cn bmRegs cpuMemEv bmRegs' :
  lts_step (bmCPU.lts cn) bmRegs (Some cpuMemEv) bmRegs' →
  bmCPU.core cn bmRegs ⊢
  requester (bmCPU.lts cn) E γ₁ {[ cpuMemEv ]} (bmCPU.core cn bmRegs').

Lemma cpu_dram_committer_intro γ₁ γ₂ E cn bmRegs bmRegs' cpuMemEv memEv :
  dual_evts cpuMemEv memEv →
  lts_step (bmCPU.lts cn) bmRegs (Some cpuMemEv) bmRegs' →
  bmCPU.core cn bmRegs ⊢
  transports (bmCPU.lts cn) bmMEM.lts γ₁ γ₂ -*
  committer bmMEM.lts E γ₂ {[ memEv ]} (bmCPU.core cn bmRegs').

```

Fig. 27. Lemmas needed to update memory invariant for memory steps.

```

Axiom IP.bm_sync_mem_step : ∀ {c vIpRegs vIpRegs' bmIpRegs} (memEv : IP.mem_event) transfMemOp,
  IP.mem_event.to_transformed_mem_op Requester memEv = Some transfMemOp →
  lts_step vCPU.IP.lts vIpRegs (Some (vCPU.IP.Mem memEv)) vIpRegs' →
  IP.related_full_states bmIpRegs vIpRegs →
  ¬IP.mem_event.trapped memEv →
  ∃ bmIpRegs',
    IP.related_full_states bmIpRegs' vIpRegs' ∧
    lts_step (bmCPU.IP.lts c) bmIpRegs (Some (bmCPU.IP.Mem memEv)) bmIpRegs'.

```

Fig. 28. The axiom required to maintain the CPU relation in a synchronous memory access step.

Here, `IP.mem_event.to_transformed_mem_op` (elided) simply translates across differences in the event label types. With this axiom, we can conclude our proof, and re-establish both the memory invariant and the CPU invariant together.

6.4 Software Emulation for the VMM's Accessing Memory

As we saw in [Section 6.3](#), hardware-virtualized accesses to memory can be handled using the ownership `space_page_mappings GST gpa_to_hpa` from the memory invariant ([Figure 25](#)). However, one physical memory page (typically 4KB in size) can be concurrently accessed not only by guest CPUs but also by guest devices and the VMM's C++ code. Some examples of reasons for these accesses include:

- Guest devices can also access memory (direct memory accesses, or DMAs). If the device is a passthrough device, the access is also hardware-virtualized, and the 2nd-stage page tables need extending to account for mapping host DMA ranges (HVs) to HPAs. If the device is a virtual device ([Section 8](#)), the VMM software-emulates the DMA by accessing the physical memory that backs up the guest DMA ranges.
- When 1st-stage address translation (translating GVA to GPA) of the guest faults, the VMM's Data Abort handler ([Section 7.2](#)) needs to emulate in software both the 1st-stage address translation and the actual

```

Definition memory_inv_v2 :=
  inv memory_ns (
    ∃ m gpa_to_hpa hva_to_hpa dma_spaces,
    (* whole bare-metal memory ownership *)
    AuthSet.frag γmem m * bm.bytes_at_auth (to_map m) * ... *

    (* mapping from guest main memory ranges to physical memory ranges,
    for hardware virtualization of CPU accesses *)
    space_page_mappings GST gpa_to_hpa *

    (* mapping from guest DMA ranges to physical memory ranges,
    for hardware virtualization of device accesses *)
    [* list] dma_space ∈ dma_spaces, space_page_mappings DMA dma_space.(mapping) *

    (* mapping from some of VMM's host virtual address (HVA) ranges to physical memory ranges,
    for software-emulated accesses from C++ code *)
    space_page_mappings HST hva_to_hpa *

    (* related hardware bytes and fictionally separated bare-metal bytes *)
    [*list] gpa ∈ dom (to_map m), ∃ n, bm.byte_at gpa 1 n * hw.byte_at (gpa_to_hpa gpa) 1 n
  ).

```

Fig. 29. Version 2 of the memory invariant: more page table mapping ownership for aliasing the physical memory.

access itself. The VMM then needs access to not only physical memory targeted by the access, but also to the 1st-stage page tables stored in guest memory.

- Furthermore, the VMM implements introspection mechanisms that intercept guest memory accesses by altering page table entries, *e.g.*, by removing read or write permissions.

To support these other accesses, we need to extend the memory invariant with ownership of other page tables, as shown in [Figure 29](#) (v2). To support different types of page table mappings, NOVA [14, §2.1] separates them into *spaces*: a guest (GST) memory space page table maps guest memory (GPA) to host memory (HPA), while a DMA space page table maps guest DMA ranges to host memory, and a host (HST) space page table maps host virtual memory (HVA) of the BlueRock stack (including the VMM) to host memory.

- The assertion `space_page_mappings DMA dma_gpa_to_hpa` is the ownership of the DMA page tables. The map `dma_gpa_to_hpa` does not necessarily include all guest ranges, only those available to guest DMA.¹⁰ As there can be multiple DMA spaces, potentially one for each device, we collect them in the memory invariant. Hardware-virtualized DMA accesses from device will use this ownership to justify their accesses. In particular, the ownership will be used to prove the asynchronous cases of `do_mem_or_mmio_read` or `do_mem_or_mmio_write`.
- The assertion `space_page_mappings HST hva_to_hpa` is the ownership of the host page tables that map some of the HVAs to HPAs. A HST space is in any case needed by the VMM to access memory, but in the memory invariant we only care about certain physical memory ranges that back up the guest memory and that are of concern to VMM C++ code. This subset of memory is captured in the map `hva_to_hpa`. The ownership of these page tables is crucial in the proofs of the Virtual Bus and the Data Abort handler ([Section 7](#)).

¹⁰In the absence of an IOMMU, bare metal devices can access the full address range; however, the BlueRock VMM can be configured to hide some memory, *e.g.*, kernel memory, from devices for added security. While this configuration breaks the bare metal property in its purest form, this restriction is compatible with modern software stacks and provides some protection of the guest from hardware devices.

Other features of the memory invariant. For the sake of the exposition, we elided many details of the memory invariant. This includes the relaxed-memory characterization of page table updates, which is highly intricate and is still work-in-progress. One important feature of the memory invariant is its support for the roundup protocol ([Appendix A](#)): the roundup when successfully executed should cease all executions of the guest VM, and consequently the roundup caller can have access to the whole guest memory, for example for introspection. The full memory invariant thus also supports temporary extraction of the physical memory ownership. Last but not least, while we believe the skeleton of the memory invariant is mostly adequate, more work is still needed to flesh out the exact resources necessary to support holes in the address space mapping to support both passthrough and virtual devices.

7 THE VIRTUAL BUS

While we often think of “memory accesses” as targeting memory, as we saw in the whole machine model ([Section 3](#)) these accesses can also target devices. In the bare-metal model, determining what component an access targets is the job of the Resolver. In the VMM implementation, this functionality is handled by a combination of the virtual bus (Vbus) and the 2nd-stage hardware page tables (discussed in [Section 6](#)). The 2nd-stage hardware page tables handle the mappings for hardware virtualized components, *i.e.*, memory and devices, while the Vbus manages software emulated devices. In this section, we discuss how the VMM virtual bus maintains its simulation with the Resolver.

7.1 The Vbus Invariant

Recall that in the bare-metal machine model, the Resolver is responsible for (i) looking up the device that serves the accessed range and (ii) updating the mapping from memory ranges to devices. There are several interesting problems about these behaviors of the Resolver.

Updates to device mappings are also memory accesses. Updates of the mapping from memory ranges to devices are done through Configuration messages, which also typically target a segment of the system memory map. For example, a PCI(e) device has multiple Base Address Registers (BARs) that store which memory ranges the device is serving, and the BARs are also mapped into system memory at some specified ranges (called the Configuration Space). CPUs can reconfigure a PCI device to serve a different range by writing to the corresponding addresses of the BARs. Note that Configuration registers (which include BARs) are part of the device, but are disjoint from device registers that serve the device’s functionalities.

The BlueRock VMM requires that any updates to device mappings by the guest must be handled by the VMM. That is, even if a device is passthrough device, its Configuration registers are not passed through, and therefore are owned by the Vbus. In other words, guest accesses to the Configuration ranges fault into the VMM, and are handled by the Vbus. The Vbus updates its C++ device mappings accordingly, and, if this affects passthrough devices, the Vbus performs the necessary Configuration updates to the underlying physical Configuration registers. In essence, the Vbus simulates the behaviors (ii) of the Resolver fully.

Lookups and updates of device mappings can happen concurrently. This means that when an access occurs to a range of memory that is being remapped, it can be the case that the access may not be served (no device found at the address), or the access is served by either the old device or the new device, depending on interleaving of the access and the Configuration access. It is worth noting that this behavior is undefined within the PCI specification, so it should not happen in practice. While we can not assume the guest is well-behaved, since that would expose a potential vulnerability within the VMM, it is okay for this path to be a slow path. The BlueRock VMM reduces the number of cases by employing a reader-writer lock around the Vbus. A lookup would need to acquire the read permission on the Vbus, while an update requires the write permission. Note that this only

```

vbusI vbus_ptr cfg :=
  vbus_ptr, _field `::Vbus::Bus::_vbus_lock` ↦ rlockIR (
    ∃(vb_map : map Range DeviceEntry) (rs : Resolver.(state)),
    (* ownership of the Resolver state from Spectra *)
    AuthSet.frag g_rs rs *
    (* C++ state stores vb_map *)
    _field `Vbus::Bus::_devices` ↦ RangeMapR vb_map *
    (* C++ state agrees with the static and dynamic topology *)
    [ cfg.(static_topo) ∪ rs.(topo) = vb_map ] *
    (* ownership of the Configuration state of dynamic devices *)
    [* list] dev ∈ rs.(topo), AuthSet.frag dev.(g_cfg) dev.(cfg)
  )

```

Fig. 30. The (simplified) simulation invariant of the virtual bus.

reduces the number of cases for accesses that fault and go through the Vbus: normal guest accesses to passthrough devices would still have arbitrary interleavings with device mapping updates. In essence, the Vbus refines the behaviors (i) and (ii) of the Resolver by reducing concurrency non-determinism.

The Resolver is not involved in an access to the static topology of the bare-metal state. However, the Vbus still needs to allow faulting accesses to look up static components, such as DRAM. That is, the Vbus also simulates the static topology of the bare-metal. Consequently, its C++ mappings for the static topology remain constant during a life cycle of the VM.

The Vbus invariant, which maintains the simulation, `vbusI` is then defined as given in [Figure 30](#) (simplified). That is:

- the simulation is protected by the Vbus’s reader-writer lock, as wrapped in the predicate `rlockIR`;
- the bare-metal Resolver state, as well as the bare-metal Configuration state of dynamic devices, are owned by the simulation;
- the C++ map `vb_map` agrees with both the static topology (`cfg.(static_topo)`), as well as the dynamic topology (`rs.(topo)`) as recorded in the Resolver state and the Configuration state of devices.

Future Work. We note that our current version of `vbusI` does not yet account for guest accesses to passthrough devices, because the simulation is simply put inside the reader-write lock, and so can only be accessible by acquiring a lock. In order to support passthrough devices, we need to relate the Resolver state as well as the bare-metal Configuration state of passthrough devices to the physical Configuration state of the physical devices. We consider the support for passthrough devices as future work.

7.2 The Verification of the Data Abort Handler

The Data Abort handler is called by the `call_portal_handler` ([Section 5.3](#)) in order to handle the specific case where a memory access from the vCPU traps due to a fault in the 2nd-stage address translation. In the common case, the handler parses the address out of the decode assist and invokes the `Vbus::Bus::access` (shown in [Figure 31](#)) to emulate the access. The code is rather simple: the lookup of the device for the target range in the Vbus’s map is performed within the reader lock. The looked up device is stored in `entry → device`, and the access is performed by calling `Vbus::Device::access` on the device.

`Vbus::Device::access` is a virtual method that is implemented by the concrete device emulator. In our on-going verification of the Vbus, we assume a virtual method specification for `Vbus::Device::access`, which is responsible for maintaining the simulation of the device, and which we will discuss in [Section 8](#). The proof then pins down to

```

1 Vbus::Err
2 Vbus::Bus::access(Vbus::Access access, const VcpuCtx& vcpu_ctx, mword addr, uint8 bytes, uint64& val) {
3     if ( __UNLIKELY__((addr + bytes <= addr)))
4         return NO_DEVICE;
5
6     Range<mword> target(addr, bytes);
7     const DeviceEntry *entry, *previous_entry = nullptr;
8
9     _vbus_lock.renter();
10    entry = lookup(addr, bytes);
11    if (entry == nullptr) {
12        _vbus_lock.rerexit();
13        return NO_DEVICE;
14    }
15
16    mword off = _absolute_access ? addr : addr - entry->begin();
17    _vbus_lock.rerexit();
18
19    Err err = entry->device->access(access, vcpu_ctx, off, bytes, val);
20
21    return err;
22 }

```

Fig. 31. The (simplified) code of Vbus::Bus::access.

the fact that, as we have the simulation within the reader lock, the C++ lookup method returns the device that matches the bare-metal device.

Future Work. Our specification and verification does not yet account for the case where Vbus::Device::access is a Configuration update, which should acquire the writer lock and update the simulation correspondingly. While such an extension may appear straightforward, we expect that it will have interesting interactions with passthrough devices. Furthermore, our specification and verification does not yet consider accesses from other devices (peer-to-peer accesses).

Data Abort due to 1st-stage address translation faults. The Data Abort handler is typically called due to 2nd-stage address translation faults, e.g., when the 1st-stage address translation succeeds to translate the guest virtual address (GVA) to the guest physical address (GPA), but the GPA is not mapped to an underlying physical machine address (HPA) in the 2nd-stage page tables. However, there are rare cases where the handler needs to handle faults at the 1st-stage address translation. In these cases, the guest is blocked on that single faulting instruction, and the Data Abort handler has to emulate the 1st-stage address translation in software, before emulating the actual access. We have provided specifications for these cases, where we have made several assumptions on the bare-metal CPU model:

- If 1st-stage page table walk faults, the bare-metal CPU should have trivial asynchronous transaction state when trying to re-execute the instruction.
- Decode assist and 1st-stage page table walk do not affect CPU internal state and transaction state.

We consider the verification of these cases in the Data Abort handler as future work.

8 DEVICES

Besides CPUs, the other major actor in the system is devices. Like CPUs, devices are active entities that can initiate memory accesses (DMAs), communicate with other devices (peer-to-peer), and can raise interrupts. However, devices can also respond to accesses from CPUs and other devices. While accesses can be through legacy mode, for example with Port I/O channels or pin interrupts, our verification efforts focus on the unified interface—that is, through the system memory map: device registers are mapped to memory (MMIO) and interrupts are also raised through memory (MSI).

As mentioned in the previous section, a device is either a *passthrough device* (hardware-virtualized) and is accessed directly by the guest through 2nd-stage hardware page tables; or a *virtual device* (software emulated) and is accessed by the guest through the VMM’s virtual bus (Vbus). [Section 8.1](#) discusses the specification and verification of VMM’s virtual devices. We consider it future work to specify and verify passthrough devices and re-locatable (dynamically mapped) devices.

8.1 The Virtual Device Interface

A VMM’s virtual device software-emulates a bare-metal device. As such, we assume an operational specification for every bare-metal device (which is also needed for passthrough devices). However, the exact properties of the device operational model are only useful for the concrete device emulator, and not to the rest of the VMM. Therefore, the VMM virtual bus implements the abstract class `Vbus::Device` and a C++ virtual method interface of `Vbus::Device::access`. The concrete device emulator will be derived from `Vbus::Device` and implement the access method according to the concrete bare-metal device model.

Recall that, the general specification assumes the ghost state `AuthSet.frag  $\gamma_{dev}$  s_dev` of the bare-metal device state from the Spectra library. The virtual interface specification simply quantifies over the bare-metal device model, while the concrete specification assumes the concrete device model.

The C++ virtual method interface for `Vbus::Device` follows the general setup of BRiCk. [Figure 32](#) shows the instantiation of the setup for `Vbus::Device::access`. The predicate interface methods `baseR` `derivedR` represents the general requirements for virtual methods:

- If `baseR` is the representation predicate of the base class, and `derivedR` is that of the derived class, then we require that we can always *borrow* the base class ownership from the derived class ownership.
- methods record the list of virtual methods and their corresponding virtual specifications. We require that for each virtual method, its virtual specification `spec` holds for the derived class ownership: `vptrI name derivedR (spec derivedR)`. The predicate `vptrI` of BRiCk requires, among many things, that the virtual method name will be dynamically dispatched to the implementation of the derived class.

Intuitively, at runtime, the caller should own the derived class instance (with `derivedR`), and calls the virtual method name on the derived instance through the lens of the base class, using the virtual specification `spec`.

For the base class `Vbus::Device`, we pick the `baseR` as `StaticR`, which contains ownership of various fields of `Vbus::Device`. We pick the methods list as `vbus_device_methods`, which includes at least `Vbus::Device::access` and its virtual specification `accessSpec`. `deriveR` will be picked by the verification of the concrete device emulator implementation.

Our virtual specification `accessSpec` for now focuses on the case where `Vbus::Device::access` is called from a vCPU `vcpu_id` (and not a device). The specification is parametric to the bare-metal device model `devLTS : Device`. Most importantly, the specification assumes from the caller the resource `devCommitter vcpu_id  $\gamma_{dev}$  sp access` off bytes value `Q`. This is an instance of the Spectra library committer ([Figure 3](#))—an atomic update that:

- requires that the verification of the virtual method implementation atomically updates the bare-metal device state by making a step (of `devLTS`) with a visible Event depending on access (*committing the event*). If access is a read, the step is a read on the bare-metal device, and similarly for a write.

```

(* general virtual interface requirements *)
Definition interface methods (baseR derivedR : cQp.t → Vbus.Device.Typ.t → Rep) : mpred :=
  (* borrowing baseR from derivedR *)
  □ (∀ (this : ptr) q dty, derivedR q dty →* (baseR q dty * (baseR q dty →* derivedR q dty))) *
  (* each virtual method satisfies virtual [spec] *)
  ([ * list] method ∈ methods, let (name, spec) := method in virtual.vptrI name derivedR (spec derivedR)).

Definition StaticR (q : cQp.t) (dty : Vbus.Device.Typ.t) : Rep :=
  structR Vbus.Device.T q * _field`::Vbus::Device::_dev_type` ↦ Vbus.Device.Typ.R (cQp.c q) dty * ...

(* virtual Vbus::Err Vbus::Device::access(Access, VcpuCtx*, Vbus::Space, mword, uint8, uint64&) *)
Definition accessSpec {devLTS : Device} (derivedR : cQp.t → Vbus.Device.Typ.t → Rep) : WpSpec_cpp :=
  \arg{access} "access" (Vaccess access)
  \arg{vcpu_ctx} "vcpu_ctx" (Vptr vcpu_ctx)
  \arg{sp} "sp" (Space.to_val (ofSpace sp))
  \arg{off} "off" (Vn off)
  \arg{bytes} "bytes" (Vn bytes)
  \arg{res} "res" (Vptr res)

  \pre{value : N}
  match access with | DataREAD ⇒ res ↦ anyR Tu64 1 | DataWRITE ⇒ res ↦ u64R 1 value end

  \prepost{vcpu_id ...} vcpu_ctx ↦ VcpuCtx.R vcpu_id ...

  \prepost{q dty} this ↦ derivedR q dty
  \pre{y_dev (Q : access_type access → mpred)} devCommitter vcpu_id y_dev sp access off bytes value Q

  \post[Vint (Err.to_Z Err.OK)]
  match access with
  | DataREAD ⇒
    ∃ (retV : N), Q (if access return access_type access then Some retV else true) * res ↦ u64R 1 retV
  | DataWRITE ⇒
    Q (if access return access_type access then None else true) * res ↦ u64R 1 value
  end.

Definition vbus_device_methods {devLTS : Device} :=
  [ ({% cpp (named "Vbus::Device::access"), accessSpec}; ... ].

```

Fig. 32. The virtual interface specification for Vbus::Device.

- requires also that the update returns the resource Q that is picked by the caller and will be returned in the post-condition.

Recall that the verification of the concrete device emulator assumes, at the beginning of the device simulation, the bare-metal device ghost state ownership, *i.e.*, $\text{AuthSet.frag } \gamma_{\text{dev}} \text{ s_dev}$ where $\text{s_dev} : \text{devLTS}(\text{state})$. When performing the update, the verification will have to maintain the device simulation.

The updates of the bare-metal device state and the driving of its steps are further complicated by the fact that an access can be asynchronous, which means that it requires at least two visible events: one to initiate the transaction and one to complete the transaction. These obligations are embedded into devCommitter , and we elide their details here.

Work-In-Progress. We have several on-going efforts in the specification and verification of virtual devices. We have developed bare-metal operational models of ARM interrupt controllers (e.g., GICD, GICV [2]) and a ARM UART device (PL011 [1]). We have given concrete C++ specifications for these virtual device emulators, and we are in the progress of verifying them against the virtual specification explained above.

9 VMM STARTUP

Up until this point, we have focused on the steady state of the VMM. In doing so, we have described the ownership decomposition and invariants used to ensure the simulations. In this section, we turn to the startup phase of the VMM. Verifying the startup logic ensures that these invariants are actually realizable. Ultimately, it is the proof of the entry point that feeds into the overall simulation proofs, and the proofs of the steady state are “merely” maintaining the invariants.

The startup logic of the VMM is, to a large extent, straightforward but verbose and relies on many parts of the ULTRA platform, e.g., getting access to system resources. However, there are two interesting aspects of the setup that warrant more discussion. The first is the concurrent startup protocol that the VMM implements in order to reduce startup latency (Section 9.1). The second addresses the parsing the guest machine configuration and connecting it to the guest specification (Section 9.2). In the latter, we discuss the approach that we take to enable some non-determinism in the specification so that the VMM can dynamically include resources that it discovers without compromising the overall specification.

9.1 The Startup Process

The VMM initialization establishes the simulations for each component. However, there are two subtleties in these components that complicate this process. First, the components are linked, e.g., devices can raise interrupts to the global interrupt controller, who can forward these interrupts to per-CPU local interrupt handlers. On its own, this linking is not too difficult. But some of these components are active, i.e., they rely on internal threads or encapsulate hardware devices that are running concurrently with the rest of the system. In order to avoid dynamic checks in the steady state, we stage the startup to ensure that invariants are completely set up before steady state execution begins, though our current setup has some limitations on this front.

We will focus on the following constraints, which are driven by a combination of hardware requirements and the VMM implementation.

- The global interrupt controller must not receive interrupts before the local interrupt controller of the target CPU is attached. This allows the C++ code of the global interrupt controller to dereference non-atomic pointers to determine the target CPU.
- Devices must not start without the global interrupt controller being able to process interrupts. This is because devices may already have pending interrupts that should be routed to the guest. Empirically, we have witnessed a number of systems where this sort of behavior is crucial to successful virtualization. Formally, this requirement follows naturally from the code and the fact that models of hardware are not limited on when they can raise interrupts.

Additionally, we would like to ensure that memory is available before devices run. Ensuring this property is subtle because hardware passthrough devices are running even before the VMM starts and devices must be added to the VMM one at a time. In discussions with systems engineers, we believe that we can weaken this property to only require certain distinguished hardware-reserved regions are accessible to devices early. Under this assumption, we believe that we can carry out the startup verification. However, currently our verification assumes that hardware devices do not access arbitrary memory (or other devices) until memory is accessible by the CPUs. This corresponds to after step (6) below (also in Figure 34).

```

void start_vmm () {
    // parse the guest machine configuration file
    Bson vmm_bson = Bson::decode();
    const UserConfig conf{BsonConfig(vmm_bson)};
    prepare_address_space_from_config(conf); // (1)
    Model::Board::init(conf); // (2)
    init_vcpus(conf) // (3)
    Model::Board::setup_passthru_devices(); // (4)
    Model::BoardImpl::late_init(); // (4)
    run_vm(conf);
    vmm_can_shutdown = true;
}

void run_vm(const UserConfig &conf) {
    // Start vCPU threads and then wait
    for(auto cpu : conf.cpus) // (5)
        Vcpu::run(cpu)
    // Initialize memory with the guest image
    Model::vm_asl → setup_guest_as(); // (6)
    // Barrier to wait for vCPUs
    Vcpu::Roundup::wait_for_all_off();
    setup_boot_args(); // (7)
    // unblock guest boot CPU
    Model::Cpu::start_cpu(0); // (7)
}

```

Fig. 33. The (simplified) VMM startup routine.

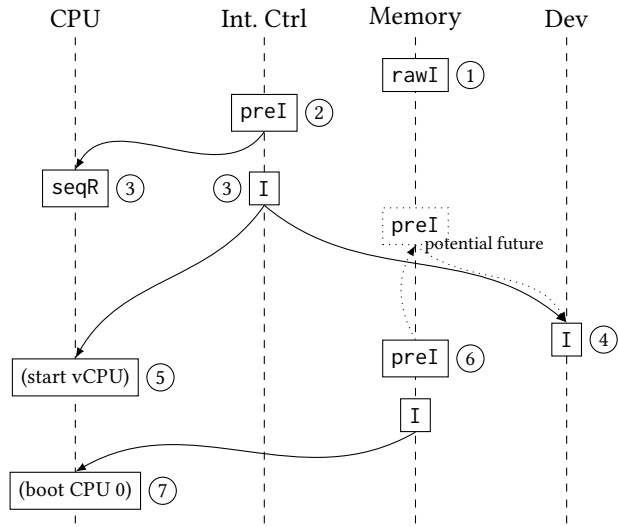


Fig. 34. The setup process of the VMM.

Figure 33 shows the (simplified) code that initializes the VMM and Figure 34 illustrates the way that the separation logic ownership evolves throughout the code. In this process, the VMM

- (1) Sets up the 2nd-stage page tables described by the user configuration (passed using BSON) using the NOVA memory space API [14, §5]. This constructs the memory invariant in phase *rawI*.
- (2) Initializes the board, including the global interrupt controller (GIC) as well as some constrained devices that are purely passive.
- (3) Creates and initializes the CPUs and connects them to the global interrupt controller. At this point, the interrupt controller can begin to process interrupts, though they will not be injected into the guest until the CPUs that they target are initialized.
- (4) Initializes passthrough and virtual devices, and gives them shared/fractional ownership of interrupt controller (now fully-initialized) and the partially constructed memory invariant. The initialization of a device is encapsulated in a generic specification that is parametric to the device’s hardware model and thus abstracts away the specifics of device initialization.
- (5) Starts executing the virtual CPUs and transfers ownership of the interrupt controller to the thread. Each of these concurrent runs initializes the registers of the vCPU then blocks on a barrier, waiting for the guest to actually boot up.
- (6) Initializes the memory contents from the guest image concurrently with the startup of the virtual CPUs. This advances the phase of the memory simulation from *preI* to *I*.
- (7) Sets up the boot arguments for the boot CPU and unblocks it, allowing it to start running guest code. This passes the knowledge that the memory is fully initialized to the vCPU. When that vCPU starts other vCPUs, it will pass this ownership along ensuring that all CPUs that run know that the memory has been initialized.

The modularity of the Iris separation logic makes it possible to capture this initialization protocol despite the fact that the initialization will create an unknown number of threads within the device initialization.

```

\pre{g_bm cfg} AuthSet.frag g_bm init_state(cfg)
\pre{(guest_fdt : Fdt.Tree.T) q_fdt fdt_uuid}
  right_to_import_from_initrd (Fdt.Tree.R q_fdt guest_fdt) fdt_uuid
\require guestfdtEncodes cfg guest_fdt
\pre ...
\post ...

start_vmm()

```

Fig. 35. The specification of `start_vmm` (excerpt) connects the specification (ghost) state (`AuthSet.frag`) to the contents of a “file” that the VMM has the right to “open” through the pure predicate `guestfdtEncodes`.

9.2 Interpreting the Guest Configuration

The VMM provides the simulation of a guest virtual machine, which shows up in our specifications as Spectra ghost state. As it starts up, the VMM also needs to read physical machine state to learn about the description of the bare-metal, *e.g.*, the number of CPUs or the amount of memory that it has. This information comes from a configuration file in BSON format provided by the user and is decoded by the VMM `start_vmm` function into the C++ object `UserConfig` (Figure 33).

Figure 35 shows an excerpt of `start_vmm` specification. We assume in the pre-condition that we own the bare-metal machine state in the initial state `init_state(cfg)` of the machine configuration `cfg`. We require that the machine configuration `cfg` is encoded as `guest_fdt` that is being stored as user-provided configuration. That is, when the `start_vmm` code parses the `guest_fdt`, it will read the machine configuration that agrees with our bare-metal machine ghost state.

Unfortunately, this simple intuition is not sufficient to support more complex situations where the VMM adds more devices to the guest based on the devices available at runtime. For example a specification where the guest should have *at least* 4GBs of RAM, but should get all of the remaining RAM on the system. In such situations, our setup of separating a machine configuration from a machine semantics gets in the way. We need to delay the choice of the machine configuration until after we have finished parsing. This was an unforeseen enhancement to the overall specification that evolved due to changing business objectives and was not planned into the initial specification.

The flexibility of separation logic, however, provides a way to *alter the startup specification* to support this change in the overall requirement. Specifically, we introduce an additional state (`Unpicked`) to the bare-metal state to allow for delayed non-deterministically picking a machine configuration that is within some constraints to the guest-provided BSON configuration (`guest_fdt`). When the VMM finally determines the final configuration `cfg` that it wants to use, it transitions the state from `Unpicked` to `Picked cfg (init_state cfg)` by showing that `cfg` is compatible with the guest-provided configuration.

One downside to this phase split is that it prevents decomposing the state until the machine configuration is picked. An alternative approach that *might* support this would require a refactoring of the machine setup to avoid the need for a separate machine configuration. With this setup, it might be possible to use the non-determinism features built into the Spectra library to perform the choice. Such a change, again *might*, simplify support for hot-plugging and hot-unplugging of devices.

10 CONCLUSION

We have presented a refinement specification for the BlueRock virtual machine monitor. The specification ensures that the VMM acts as a conformant implementation of the underlying hardware architecture and, consequently, that code running on the VMM behaves as if it is running on “bare metal”. Towards the goal of proving this

property, we have presented the architecture of our verification and how it naturally aligns with the specification and the VMM implementation. Beyond the software architecture, we discussed how the NOVA specification enables low-level reasoning about software running on top of it and how the uniform formalism of separation logic enables us to describe subtle simulation relations relating the specification to both C++ code and hardware virtualization mechanisms exposed by NOVA.

While this report represents a significant amount of work, we note that it is not a complete proof. Within the VMM, there are still unverified virtual devices, as well as more work to integrate passthrough devices and some of the presented proof sketches still need completing. In addition, the VMM sits on top of the BlueRock ULTRA platform which is not yet verified. Despite all of this, the code that we focus on represents the first line of defense against an attacker attempting to compromise the ULTRA platform, and the verification that we are undertaking represents a significant step towards securing this component.

REFERENCES

- [1] Arm Limited 2007. *PrimeCell UART (PL011) Technical Reference Manual*. Arm Limited.
- [2] Arm Limited 2013. *ARM® Generic Interrupt Controller Architecture version 2.0*. Arm Limited. Available electronically as ARM DDI 0487I.a.
- [3] Arm Limited 2022. *Arm® Architecture Reference Manual for A-profile architecture*. Arm Limited. Available electronically as ARM IHI 0048B.b.
- [4] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E. Gray, Robert M. Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, Shaked Flur, Ian Stark, Neel Krishnaswami, and Peter Sewell. 2019. ISA semantics for ARMv8-a, RISC-v, and CHERI-MIPS. *Proc. ACM Program. Lang.* 3, POPL, Article 71 (Jan. 2019), 31 pages. <https://doi.org/10.1145/3290384>
- [5] Andrew Baumann, Simon Peter, Adrian Schüpbach, Akhilesh Singhanian, Timothy Roscoe, Paul Barham, and Rebecca Isaacs. 2009. Your computer is already a distributed system. Why isn't your OS?. In *Proceedings of HotOS'09: 12th Workshop on Hot Topics in Operating Systems, May 18-20, 2009, Monte Verità, Switzerland*, Armando Fox (Ed.). USENIX Association. http://www.usenix.org/events/hotos09/tech/full_papers/baumann/baumann.pdf
- [6] Hoang-Hai Dang, David Swasey, Paolo Giarrusso, and Gregory Malecha. 2024. *A Formal Specification of the NOVA Microhypervisor's ABI*. Technical Report. BlueRock Security, Inc. https://github.com/bedrocksystems/fm-releases/tree/main/nova_interface.2024-04-16 Available by request.
- [7] Paolo G. Giarrusso, Abhishek Anand, Gregory Malecha, František Farka, and Hoang-Hai Dang. 2024. Modularizing CPU Semantics for Virtualization, Talk at PriSC 2024. <https://popl24.sigplan.org/details/prisc-2024-papers/7/Modularizing-CPU-Semantics-for-Virtualization>
- [8] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.* 28 (2018), e20. <https://doi.org/10.1017/S0956796818000151>
- [9] Gregory Malecha, Abhishek Anand, and Gordon Stewart. 2020. Towards an Axiomatic Basis for C++. In *The 11th Coq Workshop* (online).
- [10] Gregory Malecha, Jonas Hinrichsen, Gordon Stewart, Abhishek Anand, Paolo Giarrusso, Yoichi Hirai, and Hoang-Hai Dang. 2024. Spectra: a Framework for Decomposing Logical Atomicity of LTS Composition, Talk at the 4th Iris Workshop. <https://iris-project.org/workshop-2024/slides/dang.pdf>
- [11] Jeffrey C. Mogul, Andrew Baumann, Timothy Roscoe, and Livio Soares. 2011. Mind the Gap: Reconnecting Architecture and OS Research. In *13th Workshop on Hot Topics in Operating Systems, HotOS XIII, Napa, California, USA, May 9-11, 2011*, Matt Welsh (Ed.). USENIX Association. <https://www.usenix.org/conference/hotosxiii/mind-gap-reconnecting-architecture-and-os-research>
- [12] Timothy Roscoe. 2021. It's Time for Operating Systems to Rediscover Hardware. Keynote at OSDI'21, <https://www.usenix.org/conference/osdi21/presentation/fri-keynote>, <https://people.inf.ethz.ch/troscoe/pubs/2021-07-16-OSDIKeyNote-Handout.pdf>.
- [13] Udo Steinberg. 2023. NOVA Microhypervisor: Feature Update, Talk at FOSDEM 2023. <https://hypervisor.org/nova-fosdem-2023.pdf>
- [14] Udo Steinberg. 2024. NOVA Microhypervisor Interface Specification (July 27, 2024). <https://github.com/udosteinberg/NOVA/blob/e7fc14db7f191cb1878754a4705df93422410ccc/doc/specification.pdf>
- [15] Udo Steinberg and Bernhard Kauer. 2010. NOVA: A Microhypervisor-Based Secure Virtualization Architecture. In *Proceedings of the 5th European Conference on Computer Systems (Paris, France) (EuroSys '10)*. Association for Computing Machinery, New York, NY, USA, 209–222. <https://doi.org/10.1145/1755913.1755935>
- [16] Aaron Turon, Derek Dreyer, and Lars Birkedal. 2013. Unifying refinement and hoare-style reasoning in a logic for higher-order concurrency. In *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming (Boston, Massachusetts, USA) (ICFP '13)*. Association for Computing Machinery, New York, NY, USA, 377–390. <https://doi.org/10.1145/2500365.2500600>

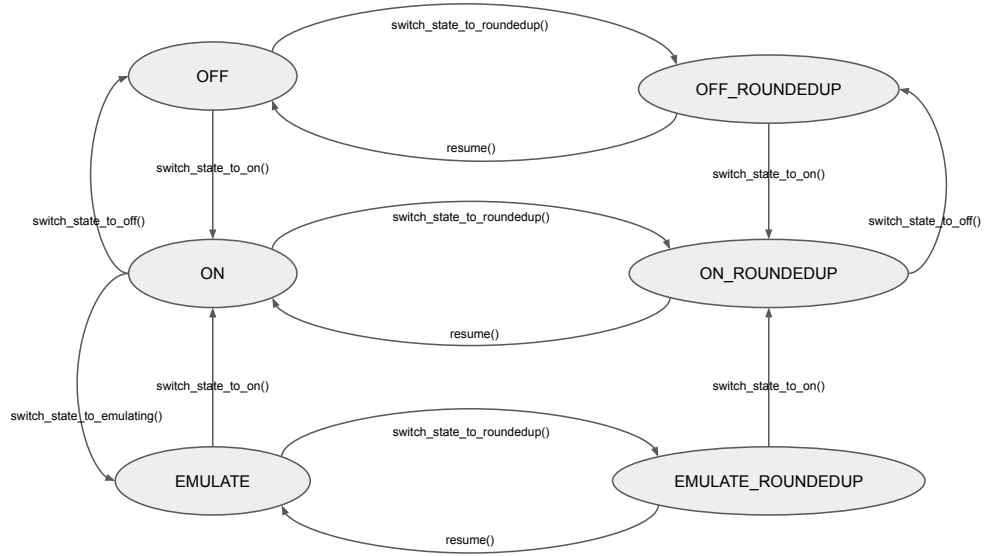


Fig. 36. High-level state transition system of vCPUs concerning roundup.

A THE ROUNDUP PROTOCOL

The VMM implementation provides a roundup mechanism `vcpu_roundup` to stop the execution of vCPUs. During the execution, a roundup request may be issued, from the lifecycle module, the VMM’s command-line interface, or from the guest code itself (through virtual firmware). A completed roundup will have stopped the execution of all vCPUs, after which one can inspect the state of the VM, or the VM can be restarted or shut down.

The main challenge in the implementation and the verification of the roundup logic is its concurrency protocol. Roundup requests can be issued concurrently from multiple participants while the vCPUs are executing using software emulation or hardware virtualization. Each roundup request signals the vCPUs to stop executing, and then waits for them to actually stop in order to complete the roundup. After all parties have acknowledged the roundup, the caller knows that no other CPUs are executing which allows it to ensure that memory that is not accessible by devices is in a stable state. When the initiator of the roundup is complete, it can release the vCPUs enabling them to resume execution.

The roundup protocol needs to account for the following interactions:

- A roundup request can be issued by a vCPU or an external monitor.
- A roundup request can be issued during boot time: once a vCPU has finished starting up and is running, it can issue a request while other vCPUs are still starting up.
- Multiple roundup requests need to be served one by one.

A.1 States of vCPUs

The high-level state transition system of a vCPU during its execution is given in [Figure 36](#). The three main states are OFF, ON, and EMULATE. In OFF, the vCPU is not running. From OFF, the state can be turned to ON, during which the vCPU is running in hardware-virtualized mode. From ON, the state can be turned to OFF, or to EMULATE: the vCPU is switched from executing in hardware to executing in software-emulated mode—VMM code emulates the vCPU instructions. Once emulation finishes, the state is switched back to ON (hardware virtualization).

Each of the main states is accompanied by a roundup state:

- When a roundup is started, if the vCPU is in OFF state, it is turned into OFF_ROUNDEDUP state. The roundup process also asks NOVA to recall the vCPU, which means that the vCPU is blocked from executing. The roundup considers the vCPU has stopped all execution.
- If the vCPU is in ON state, the process switches the vCPU state to ON_ROUNDEDUP, and asks NOVA to recall the vCPU, which is being executed in hardware. Once NOVA has recalled the vCPU, the vCPU is blocked from executing—it has to wait to be resumed. The process considers the vCPU has stopped all execution.
- If the vCPU is in EMULATE state, the process switches the vCPU state to EMULATE_ROUNDEDUP, and asks NOVA to recall the vCPU. Then there are two sub cases here:
 - The current vCPU’s emulation is also waiting to perform its own roundup, which will be run after the current roundup process. The current process considers that the vCPU has stopped all execution.
 - The current vCPU’s emulation is not waiting for a roundup. Then the roundup process waits for the vCPU emulation to finish, at which point the emulation will switch the state from EMULATE_ROUNDEDUP to ON_ROUNDEDUP. As NOVA has recalled the vCPU, the vCPU is immediately blocked from executing. The process only then considers the vCPU has stopped all execution.

Once the process has finished waiting for vCPUs in emulation, the roundup is completed. The client that calls the roundup can then access the VM’s resources, as no vCPU is executing. Afterwards, the client resumes the executions of the vCPUs. The resume process will turn the state from OFF_ROUNDEDUP, ON_ROUNDEDUP, or EMULATE_ROUNDEDUP respectively to OFF, ON, or EMULATE. For a vCPU that is waiting for its turn to perform its own roundup, the transition is from EMULATE_ROUNDEDUP to EMULATE. [Figure 37](#) provides an example run of the roundup process.

Ordering roundup requests by a mutex. As there can be multiple roundup requests at the same time, the requests are served following the order provided by a mutex. Only one roundup client can be served at a time—the one that has acquired the mutex. The client releases the mutex when resuming the vCPUs, after which other clients can acquire the mutex. If one such client is a vCPU, after transitioning from EMULATE_ROUNDEDUP to EMULATE, the vCPU can try again to acquire the mutex.

Waiting for emulations to end with a (global) signal. The roundup process waits for on-going emulations to end using a signal (`wait_for_emulation_end()`). Each emulation is marked with a block between `begin_emulation()` and `end_emulation()`, and the last emulation’s `end_emulation()` will raise the signal.

Waiting for resuming with a (per-vCPU) signal. Reversely, each vCPU emulation’s `begin_emulation()` waits for the resume signal, which is raised by the resume process for each vCPU. Once the vCPU has been recalled by NOVA, it will be run in VMM, which enters EMULATION mode with `begin_emulation()`, and therefore it is blocked until being resumed.

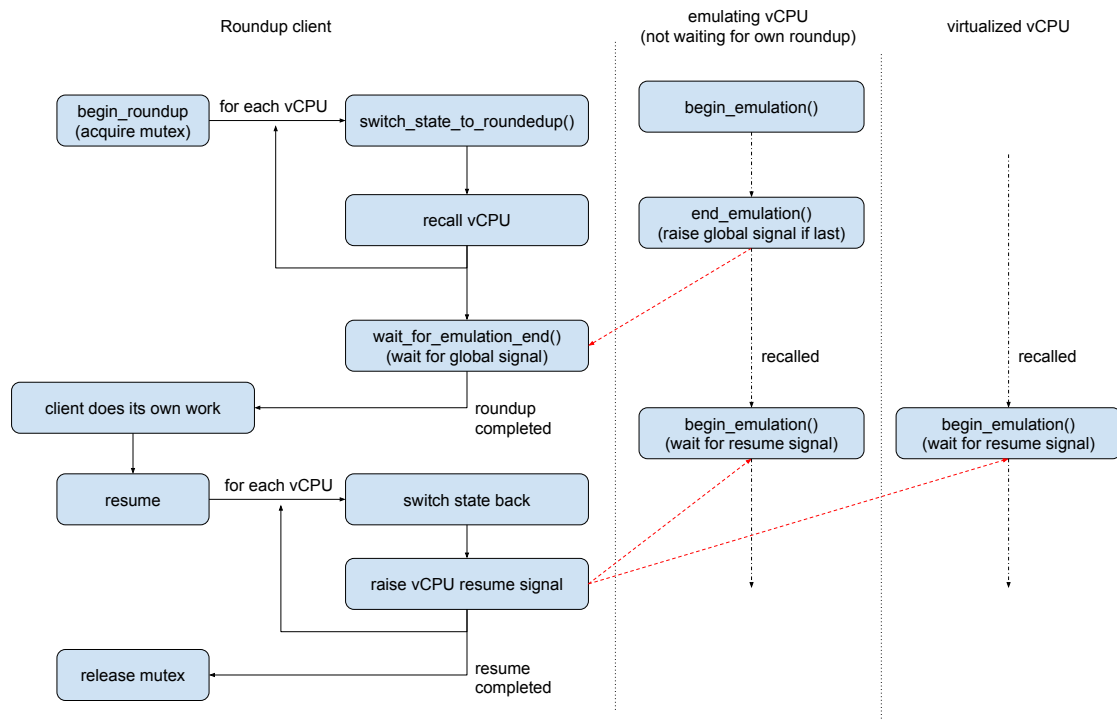


Fig. 37. An example run of the roundup process.